Week1_Gueorgui_Poklitar

June 28, 2026

[1]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn statsmodels kagglehub
```

1 Week 1 - Linear Regression 1

[2]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn statsmodels kagglehub
```

[3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures, StandardScaler,␣
↪OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import r2_score, root_mean_squared_error

from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

print("All imports successful.")
```

```
All imports successful.
```

1.1 1. Data Loading & Cleaning

[4]:
```
#Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")
```

```python
#Cleaning
df = df.drop_duplicates()

# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
 ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")
```

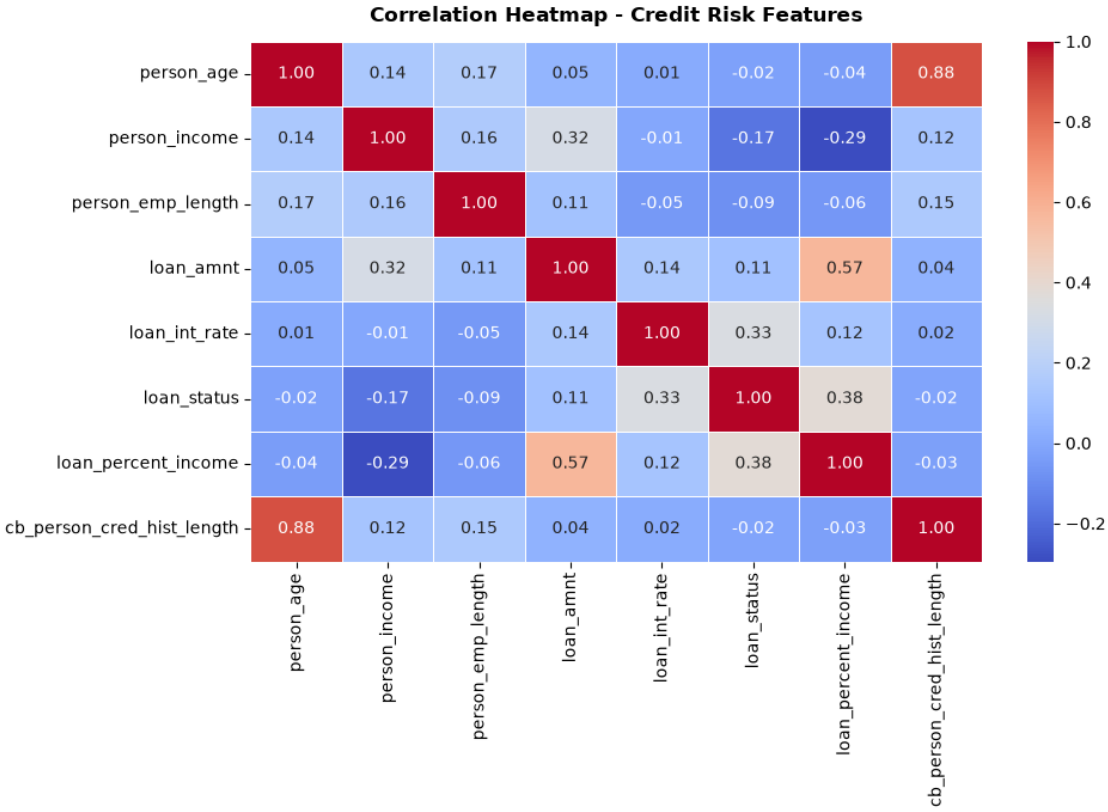
```
Downloading to /home/codespace/.cache/kagglehub/datasets/laotse/credit-risk-
dataset/1.archive…
```

```
100%|      | 368k/368k [00:00<00:00, 28.7MB/s]
```

```
Extracting files…
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate (loan_status=1): 21.87%

Column types:
person_age                      int64
person_income                   int64
person_home_ownership             str
person_emp_length             float64
loan_intent                       str
loan_grade                        str
loan_amnt                       int64
loan_int_rate                 float64
loan_status                     int64
loan_percent_income           float64
cb_person_default_on_file         str
cb_person_cred_hist_length      int64
dtype: object
```

1.2 2. Exploratory Baseline: Correlation & Feature Context

[5]:
```
numeric_cols = ['person_age', 'person_income', 'person_emp_length',
                'loan_amnt', 'loan_int_rate', 'loan_status',
                'loan_percent_income', 'cb_person_cred_hist_length']

corr = df[numeric_cols].corr()

fig, ax = plt.subplots(figsize=(10, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, ax=ax)
ax.set_title('Correlation Heatmap - Credit Risk Features', fontweight='bold',␣
 ↪pad=12)
plt.tight_layout()
plt.show()

print("Key correlations with loan_int_rate:")
print(corr['loan_int_rate'].sort_values(ascending=False).to_string())
print()
print("Interpretation: loan_int_rate correlates most strongly with␣
 ↪loan_percent_income")
print("and loan_amnt among the numeric features. The strongest single driver -␣
 ↪loan_grade -")
print("is categorical and does not appear here; it enters the model in Section␣
 ↪3.")
```




```
Key correlations with loan_int_rate:
loan_int_rate                 1.000000
loan_status                   0.334110
loan_amnt                     0.142399
loan_percent_income           0.120515
cb_person_cred_hist_length    0.015048
person_age                    0.010860
person_income                -0.005607
person_emp_length            -0.053000

Interpretation: loan_int_rate correlates most strongly with loan_percent_income
and loan_amnt among the numeric features. The strongest single driver -
loan_grade -
is categorical and does not appear here; it enters the model in Section 3.
```

1.3 3. Categorical & Continuous Features: Baseline OLS with Full Pipeline

[6]:
```
# Define features
categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']
numeric_features = ['person_income', 'person_age', 'person_emp_length',
```

```
                    'loan_amnt', 'loan_percent_income']

X = df[categorical_features + numeric_features].copy()
y = df['loan_int_rate'].copy()

mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Build preprocessing pipeline: scale numerics, one-hot encode categoricals
preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])

ols_pipeline = Pipeline(steps=[
    ('prep', preprocessor),
    ('model', LinearRegression())
])

ols_pipeline.fit(X_train, y_train)
preds_ols = ols_pipeline.predict(X_test)

r2_ols   = r2_score(y_test, preds_ols)
rmse_ols = root_mean_squared_error(y_test, preds_ols)

print("Baseline OLS (categorical + continuous features)")
print(f"  Test R-squared : {r2_ols:.4f}")
print(f"  Test RMSE      : {rmse_ols:.4f} percentage points")
print()
print("Interpretation: A high R-squared here is driven almost entirely by␣
 ↪loan_grade,")
print("which the lender assigns in lockstep with the rate. The continuous␣
 ↪features")
print("(income, age, loan size) contribute the remaining, smaller slice of␣
 ↪explained")
print("variance. Week 3 revisits this exact target WITHOUT loan_grade to force␣
 ↪an honest model.")
```

```
Baseline OLS (categorical + continuous features)
  Test R-squared : 0.9099
  Test RMSE      : 0.9678 percentage points
```

Interpretation: A high R-squared here is driven almost entirely by loan_grade, which the lender assigns in lockstep with the rate. The continuous features (income, age, loan size) contribute the remaining, smaller slice of explained variance. Week 3 revisits this exact target WITHOUT loan_grade to force an honest model.

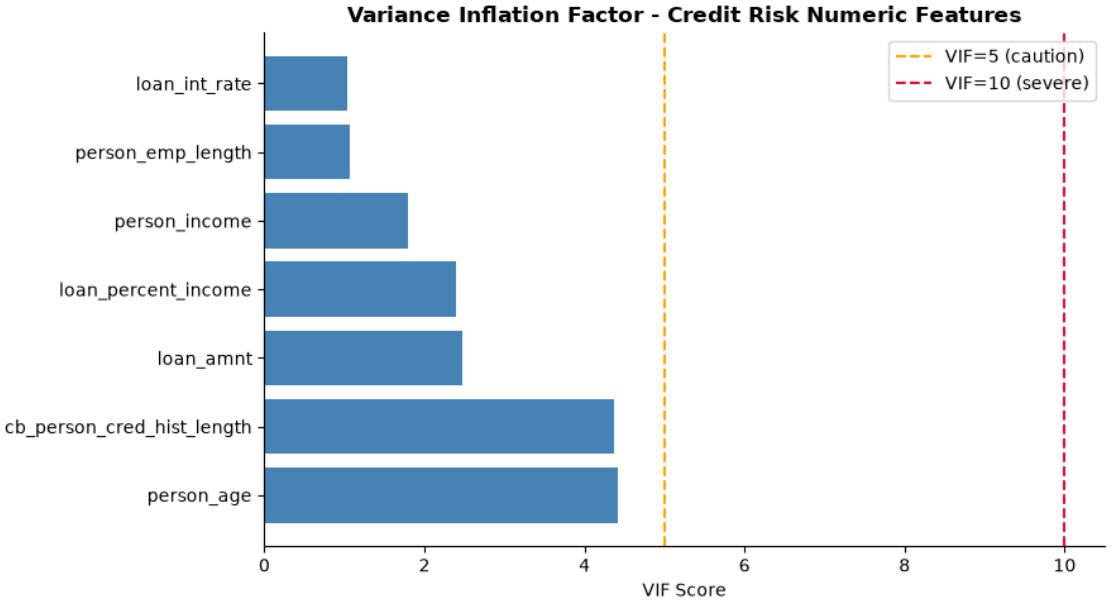
1.4 4. Multicollinearity & Variance Inflation Factor (VIF)

[ ]:
```
#VIF is computed on numeric-only features
vif_features = ['person_age', 'person_income', 'person_emp_length',
                'loan_amnt', 'loan_percent_income',
                'cb_person_cred_hist_length', 'loan_int_rate']

df_vif = df[vif_features].dropna()
X_vif  = add_constant(df_vif)

vif_data = pd.DataFrame({
    'Feature': X_vif.columns,
    'VIF':     [variance_inflation_factor(X_vif.values, i)
                for i in range(X_vif.shape[1])]
}).query("Feature != 'const'").sort_values('VIF', ascending=False)

print(vif_data.to_string(index=False))
print()
print("Finding: person_age and cb_person_cred_hist_length show elevated VIF␣
 ↪because")
print("they are nearly redundant (credit history length grows mechanically with␣
 ↪age).")
print("In all downstream models, cb_person_cred_hist_length is dropped to␣
 ↪avoid")
print("coefficient instability - this is the rule established here and reused␣
 ↪in Weeks 2-3.")

#VIF Bar Chart
fig, ax = plt.subplots(figsize=(9, 5))
colors_vif = ['crimson' if v > 10 else 'orange' if v > 5 else 'steelblue'
              for v in vif_data['VIF']]
ax.barh(vif_data['Feature'], vif_data['VIF'], color=colors_vif)
ax.axvline(5,  color='orange', linestyle='--', linewidth=1.5, label='VIF=5␣
 ↪(caution)')
ax.axvline(10, color='crimson', linestyle='--', linewidth=1.5, label='VIF=10␣
 ↪(severe)')
ax.set_xlabel('VIF Score')
ax.set_title('Variance Inflation Factor - Credit Risk Numeric Features',␣
 ↪fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
```

```
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()
```

```
                  Feature      VIF
               person_age 4.421454
cb_person_cred_hist_length 4.375880
              loan_amnt 2.471783
      loan_percent_income 2.402678
            person_income 1.799407
        person_emp_length 1.067159
            loan_int_rate 1.028640
```

Finding: person_age and cb_person_cred_hist_length show elevated VIF because they are nearly redundant (credit history length grows mechanically with age). In all downstream models, cb_person_cred_hist_length is dropped to avoid coefficient instability - this is the rule established here and reused in Weeks 2-3.



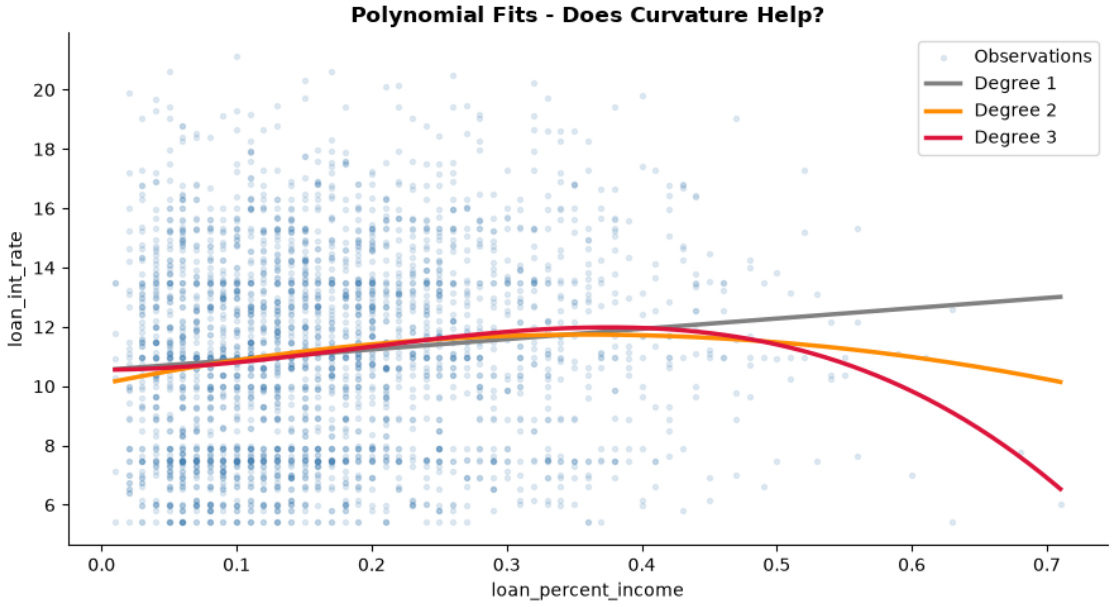
1.5 5. Polynomial Terms

[ ]:
```
#Using the two most rate-relevant numeric features for a clean polynomial test
poly_features = ['loan_percent_income', 'loan_amnt']

df_poly = df[poly_features + ['loan_int_rate']].dropna()
X_poly_raw = df_poly[poly_features]
y_poly     = df_poly['loan_int_rate']
```

```
X_poly_train, X_poly_test, yp_train, yp_test = train_test_split(
    X_poly_raw, y_poly, test_size=0.2, random_state=42
)

results_poly = []
for deg in [1, 2, 3]:
    pipe = Pipeline([
        ('poly', PolynomialFeatures(degree=deg, include_bias=False)),
        ('scaler', StandardScaler()),
        ('model', LinearRegression())
    ])
    pipe.fit(X_poly_train, yp_train)
    preds_p = pipe.predict(X_poly_test)
    results_poly.append({
        'Degree': deg,
        'R-squared': round(r2_score(yp_test, preds_p), 4),
        'RMSE': round(root_mean_squared_error(yp_test, preds_p), 4),
        'N Features': PolynomialFeatures(degree=deg).fit(X_poly_raw).
 ↪n_output_features_
    })

poly_df = pd.DataFrame(results_poly)
print("Polynomial Regression - loan_percent_income & loan_amnt to␣
 ↪loan_int_rate")
print(poly_df.to_string(index=False))

#Visual: scatter + polynomial fit (1D: loan_percent_income only)
df_1d = df[['loan_percent_income','loan_int_rate']].dropna()
df_1d = df_1d[df_1d['loan_percent_income'] < 0.8]
sample_1d = df_1d.sample(min(3000, len(df_1d)), random_state=42)

x_range = np.linspace(sample_1d['loan_percent_income'].min(),
                      sample_1d['loan_percent_income'].max(), 300).
 ↪reshape(-1,1)

fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(sample_1d['loan_percent_income'], sample_1d['loan_int_rate'],
           alpha=0.15, s=8, color='steelblue', label='Observations')

colors_deg = ['gray', 'darkorange', 'crimson']
for deg, col in zip([1, 2, 3], colors_deg):
    pipe_1d = Pipeline([
        ('poly', PolynomialFeatures(degree=deg, include_bias=False)),
        ('model', LinearRegression())
    ])
    pipe_1d.fit(sample_1d[['loan_percent_income']], sample_1d['loan_int_rate'])
```

```
    ax.plot(x_range, pipe_1d.predict(x_range), color=col, linewidth=2.5,
            label=f'Degree {deg}')

ax.set_xlabel('loan_percent_income')
ax.set_ylabel('loan_int_rate')
ax.set_title('Polynomial Fits - Does Curvature Help?', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: The marginal R-squared gain from degree 2 or 3 is tiny.␣
 ↪The")
print("rate-vs-debt-burden relationship is essentially linear; added curvature␣
 ↪mostly")
print("fits noise. This justifies keeping linear terms in the main model and␣
 ↪treating")
print("polynomial expansion as a tested-and-rejected hypothesis, not a default.
 ↪")
```

```
Polynomial Regression - loan_percent_income & loan_amnt to loan_int_rate
 Degree  R-squared   RMSE  N Features
      1     0.0220 3.1888           3
      2     0.0270 3.1806           6
      3     0.0372 3.1640          10
```



Interpretation: The marginal R-squared gain from degree 2 or 3 is tiny. The rate-vs-debt-burden relationship is essentially linear; added curvature mostly fits noise. This justifies keeping linear terms in the main model and treating polynomial expansion as a tested-and-rejected hypothesis, not a default.

1.6 6. Interaction Terms

[ ]:
```
df_int = df[['person_income','person_emp_length','loan_int_rate']].dropna().
 ↪copy()

#Manually create interaction term for transparency
df_int['income_x_emp'] = df_int['person_income'] * df_int['person_emp_length']

Xi_train, Xi_test, yi_train, yi_test = train_test_split(
    df_int[['person_income','person_emp_length','income_x_emp']],
    df_int['loan_int_rate'], test_size=0.2, random_state=42
)

results_int = []
for label, feats in [('No Interaction', ['person_income','person_emp_length']),
                     ('With Interaction',␣
 ↪['person_income','person_emp_length','income_x_emp'])]:
    sc_i = StandardScaler()
    X_tr_i = sc_i.fit_transform(Xi_train[feats])
    X_te_i = sc_i.transform(Xi_test[feats])
    lr_i = LinearRegression().fit(X_tr_i, yi_train)
    preds_i = lr_i.predict(X_te_i)
    results_int.append({
        'Model': label,
        'R-squared': round(r2_score(yi_test, preds_i), 4),
        'RMSE': round(root_mean_squared_error(yi_test, preds_i), 4)
    })
    if label == 'With Interaction':
        print("Coefficients (standardized):")
        for name, c in zip(feats, lr_i.coef_):
            print(f"  {name:<30}: {c:.5f}")
        print(f"  {'Intercept':<30}: {lr_i.intercept_:.4f}")

print()
print(pd.DataFrame(results_int).to_string(index=False))
print()
print("Interpretation: If income_x_emp carried a meaningful coefficient, the␣
 ↪marginal")
print("effect of income on rate would depend on employment stability. A small␣
 ↪interaction")
```

```
print("coefficient and a flat R-squared confirm that income and employment act␣
 ↪mostly")
print("independently for rate-setting - so the simpler additive model is␣
 ↪preferred.")
```

```
Coefficients (standardized):
  person_income                  : -0.11499
  person_emp_length              : -0.27238
  income_x_emp                   : 0.19563
  Intercept                      : 11.0270

           Model  R-squared   RMSE
  No Interaction     0.0036 3.2187
With Interaction     0.0034 3.2190

Interpretation: If income_x_emp carried a meaningful coefficient, the marginal
effect of income on rate would depend on employment stability. A small
interaction
coefficient and a flat R-squared confirm that income and employment act mostly
independently for rate-setting - so the simpler additive model is preferred.
```

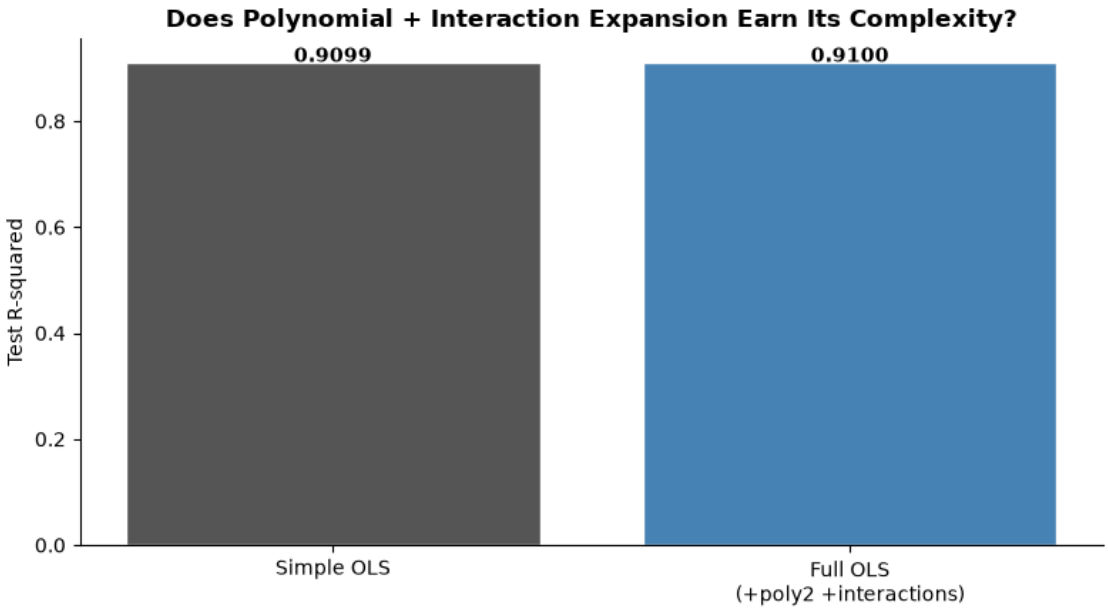
1.7 7. Bringing It Together: OLS with Polynomial + Interaction + Categorical

[10]:
```
# A single pipeline that combines everything from Sections 3-6:
#   - scaled continuous features WITH degree-2 polynomial + interaction␣
 ↪expansion
#   - one-hot encoded categoricals
#   - cb_person_cred_hist_length deliberately excluded (VIF rule from Section 4)
poly_num = ['person_income', 'person_age', 'person_emp_length',
            'loan_amnt', 'loan_percent_income']
cat_feats = ['loan_intent', 'loan_grade', 'person_home_ownership',␣
 ↪'cb_person_default_on_file']

X = df[poly_num + cat_feats].copy()
y = df['loan_int_rate'].copy()
mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,␣
 ↪random_state=42)

num_poly_pipe = Pipeline([
    ('poly', PolynomialFeatures(degree=2, include_bias=False,␣
 ↪interaction_only=False)),
    ('scale', StandardScaler())
])
full_prep = ColumnTransformer([
```

```
    ('numpoly', num_poly_pipe, poly_num),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪cat_feats)
])

models = {
    'Simple OLS (linear, no poly)': Pipeline([
        ('prep', ColumnTransformer([
            ('num', StandardScaler(), poly_num),
            ('cat', OneHotEncoder(handle_unknown='ignore',␣
 ↪sparse_output=False), cat_feats)])),
        ('lr', LinearRegression())]),
    'Full OLS (+ poly2 + interactions)': Pipeline([
        ('prep', full_prep), ('lr', LinearRegression())]),
}

rows = []
for name, m in models.items():
    m.fit(X_train, y_train)
    p = m.predict(X_test)
    cv = cross_val_score(m, X_train, y_train, cv=5, scoring='r2')
    rows.append({'Model': name,
                 'Test R-squared': round(r2_score(y_test, p), 4),
                 'Test RMSE': round(root_mean_squared_error(y_test, p), 4),
                 'CV R-squared Mean': round(cv.mean(), 4),
                 'CV R-squared Std': round(cv.std(), 4)})

compare = pd.DataFrame(rows)
print(compare.to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.bar(['Simple OLS', 'Full OLS\n(+poly2 +interactions)'], compare['Test␣
 ↪R-squared'],
       color=['#555555', 'steelblue'], edgecolor='white')
for i, v in enumerate(compare['Test R-squared']):
    ax.text(i, v + 0.002, f'{v:.4f}', ha='center', fontsize=10,␣
 ↪fontweight='bold')
ax.set_ylabel('Test R-squared')
ax.set_title('Does Polynomial + Interaction Expansion Earn Its Complexity?',␣
 ↪fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
```

```
print("What this means for the Chief Risk Officer: the elaborate feature␣
  ↪expansion buys")
print("almost nothing over the simple linear model. The honest recommendation␣
  ↪is to keep")
print("the parsimonious specification - it is easier to audit, explain to␣
  ↪regulators, and")
print("less prone to overfitting. Complexity must justify itself; here it does␣
  ↪not.")
```

```
                          Model  Test R-squared  Test RMSE  CV R-squared Mean
CV R-squared Std
     Simple OLS (linear, no poly)          0.9099     0.9678             0.9115
0.001
Full OLS (+ poly2 + interactions)          0.9100     0.9671             0.9116
0.001
```




```
What this means for the Chief Risk Officer: the elaborate feature expansion buys
almost nothing over the simple linear model. The honest recommendation is to
keep
the parsimonious specification - it is easier to audit, explain to regulators,
and
less prone to overfitting. Complexity must justify itself; here it does not.
```

1.7.1 Week 1 Takeaways and the Bridge to Week 2

What this week established: - A reusable cleaning recipe (median imputation, grade-matched rate imputation, removal of logical impossibilities) that every later notebook inherits unchanged.

- The OLS baseline for predicting `loan_int_rate`, and the finding that `loan_grade` dominates explained variance because the lender assigns rate *from* grade. - A multicollinearity rule: `cb_person_cred_hist_length` is dropped because it is nearly collinear with `person_age` (high VIF). - Tested-and-rejected complexity: neither polynomial terms nor an income x employment interaction meaningfully improve fit. The data want a simple, additive linear model.

Week2_Gueorgui Poklitar

June 28, 2026

1 Week 2 - Linear Regression 2

[1]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn statsmodels kagglehub
```

[3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.linear_model import (
    LinearRegression, Lasso, Ridge, ElasticNet,
    LassoCV, RidgeCV, ElasticNetCV
)
from sklearn.preprocessing import (
    PolynomialFeatures, StandardScaler, OneHotEncoder
)
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import r2_score, root_mean_squared_error

from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.tools.tools import add_constant

print("All imports successful.")
```

```
All imports successful.
```

1.1 1. Data Loading & Cleaning

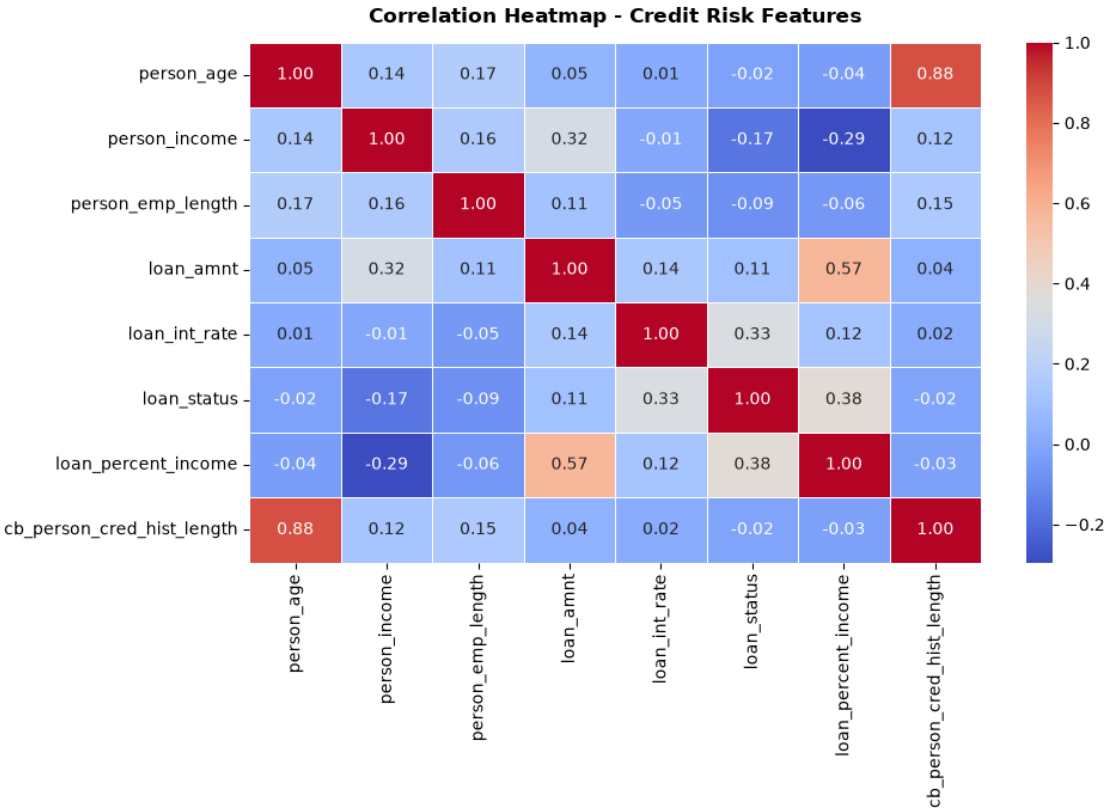
[4]:
```
#Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")
```

```python
print(f"Raw shape: {df.shape}")

#Cleaning
df = df.drop_duplicates()

# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
  ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate (loan_status=1): 21.87%

Column types:
person_age                    int64
person_income                 int64
person_home_ownership           str
person_emp_length           float64
loan_intent                     str
loan_grade                      str
loan_amnt                     int64
loan_int_rate               float64
loan_status                   int64
loan_percent_income         float64
cb_person_default_on_file       str
cb_person_cred_hist_length    int64
dtype: object
```

1.2 2. Exploratory Baseline: Correlation & Feature Context

[5]:
```
numeric_cols = ['person_age', 'person_income', 'person_emp_length',
                'loan_amnt', 'loan_int_rate', 'loan_status',
                'loan_percent_income', 'cb_person_cred_hist_length']

corr = df[numeric_cols].corr()

fig, ax = plt.subplots(figsize=(10, 7))
sns.heatmap(corr, annot=True, fmt='.2f', cmap='coolwarm',
            linewidths=0.5, ax=ax)
ax.set_title('Correlation Heatmap - Credit Risk Features', fontweight='bold',␣
 ↪pad=12)
plt.tight_layout()
plt.show()

#Key findings to note:
print("Key correlations with loan_int_rate:")
print(corr['loan_int_rate'].sort_values(ascending=False).to_string())
```




```
Key correlations with loan_int_rate:
loan_int_rate                 1.000000
```

```
loan_status                   0.334110
loan_amnt                     0.142399
loan_percent_income           0.120515
cb_person_cred_hist_length    0.015048
person_age                    0.010860
person_income                -0.005607
person_emp_length            -0.053000
```

1.3 3. Categorical & Continuous Features: Baseline OLS with Full Pipeline

[6]:
```
#   Define features
categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income']

X = df[categorical_features + numeric_features].copy()
y = df['loan_int_rate'].copy()

# Drop rows with any remaining NaN in X
mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

#   Build preprocessing pipeline
preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])

ols_pipeline = Pipeline(steps=[
    ('prep', preprocessor),
    ('model', LinearRegression())
])

ols_pipeline.fit(X_train, y_train)
preds_ols = ols_pipeline.predict(X_test)

r2_ols   = r2_score(y_test, preds_ols)
rmse_ols = root_mean_squared_error(y_test, preds_ols)

print("Baseline OLS (categorical + continuous features)")
print(f"  Test R-squared  : {r2_ols:.4f}")
```

```
print(f"  Test RMSE: {rmse_ols:.4f}")
print()
print("Interpretation: R-squared ~0.91 means loan_grade drives most of the")
print("variance in interest rate - rates are directly assigned by grade.")
print("The remaining ~9% of variance is where income, age, and loan")
print("characteristics add predictive value beyond the grade alone.")
```

```
Baseline OLS (categorical + continuous features)
  Test R-squared  : 0.9099
  Test RMSE: 0.9678

Interpretation: R-squared ~0.91 means loan_grade drives most of the
variance in interest rate - rates are directly assigned by grade.
The remaining ~9% of variance is where income, age, and loan
characteristics add predictive value beyond the grade alone.
```

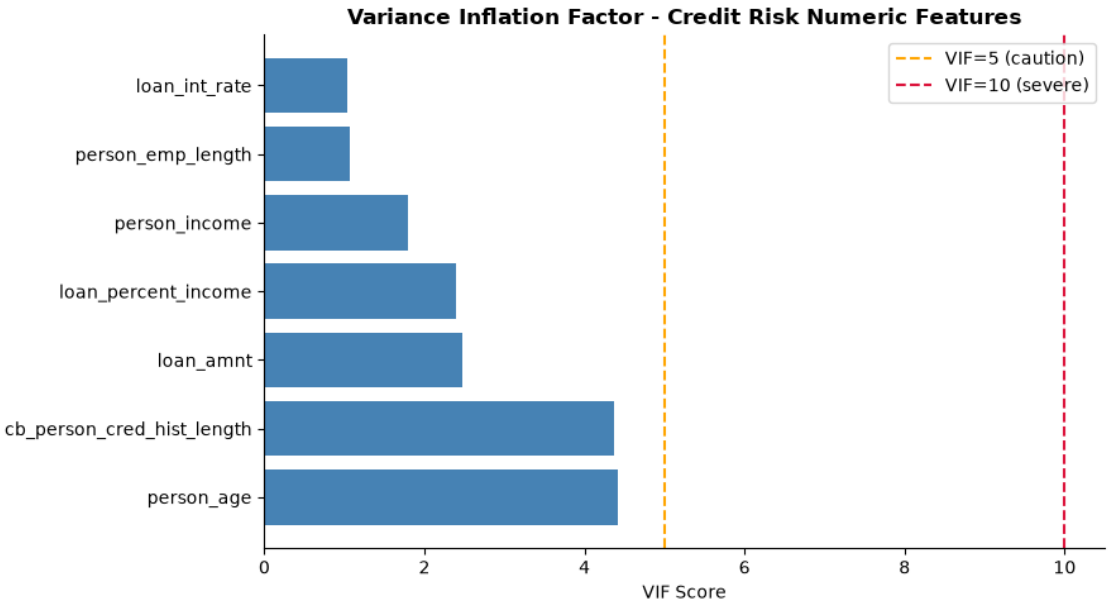
1.4 4. Multicollinearity & Variance Inflation Factor (VIF)

[7]:
```
# VIF is computed on numeric-only features (no dummies for this diagnostic)
vif_features = ['person_age', 'person_income', 'person_emp_length',
                'loan_amnt', 'loan_percent_income',
                'cb_person_cred_hist_length', 'loan_int_rate']

df_vif = df[vif_features].dropna()
X_vif  = add_constant(df_vif)

vif_data = pd.DataFrame({
    'Feature': X_vif.columns,
    'VIF':     [variance_inflation_factor(X_vif.values, i)
                for i in range(X_vif.shape[1])]
}).query("Feature != 'const'").sort_values('VIF', ascending=False)

print(vif_data.to_string(index=False))
print()
print("Finding: person_age and cb_person_cred_hist_length show elevated VIF")
print("because they are nearly redundant (r=0.88). In all downstream models,")
print("cb_person_cred_hist_length is dropped to avoid coefficient instability.")

#VIF Bar Chart
fig, ax = plt.subplots(figsize=(9, 5))
colors_vif = ['crimson' if v > 10 else 'orange' if v > 5 else 'steelblue'
              for v in vif_data['VIF']]
ax.barh(vif_data['Feature'], vif_data['VIF'], color=colors_vif)
ax.axvline(5,  color='orange', linestyle='--', linewidth=1.5, label='VIF=5␣
 ↪(caution)')
ax.axvline(10, color='crimson', linestyle='--', linewidth=1.5, label='VIF=10␣
 ↪(severe)')
```

```
ax.set_xlabel('VIF Score')
ax.set_title('Variance Inflation Factor - Credit Risk Numeric Features',␣
 ↪fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()
```

Feature	VIF
person_age	4.421454
cb_person_cred_hist_length	4.375880
loan_amnt	2.471783
loan_percent_income	2.402678
person_income	1.799407
person_emp_length	1.067159
loan_int_rate	1.028640

Finding: person_age and cb_person_cred_hist_length show elevated VIF because they are nearly redundant (r=0.88). In all downstream models, cb_person_cred_hist_length is dropped to avoid coefficient instability.

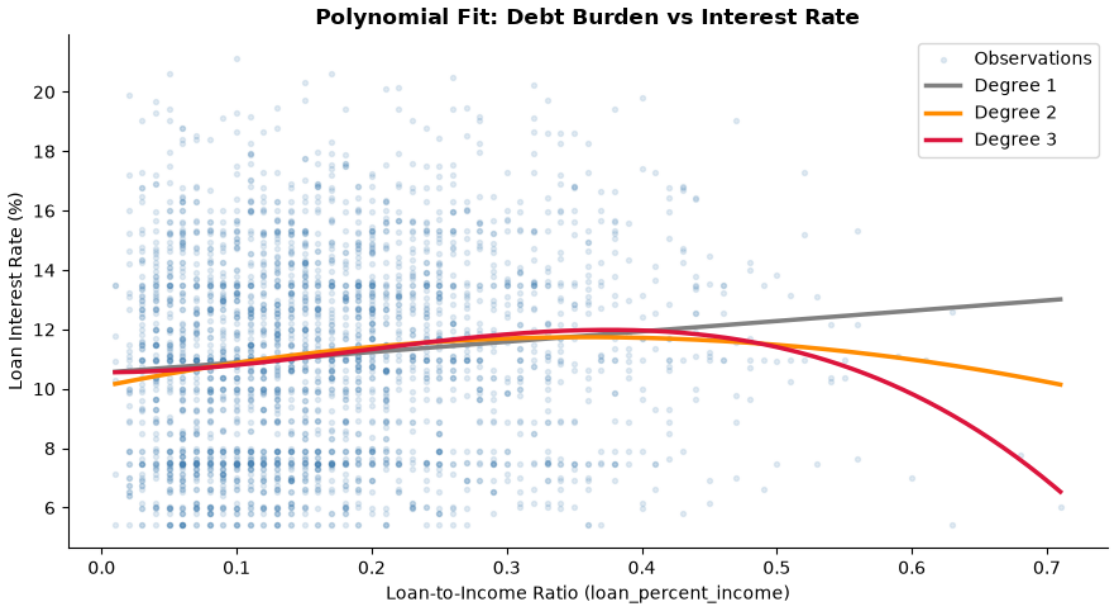


1.5 5. Polynomial Terms

[8]:
```
# Using only the two most correlated numeric features for a clean polynomial␣
 ↪test
poly_features = ['loan_percent_income', 'loan_amnt']

df_poly = df[poly_features + ['loan_int_rate']].dropna()
X_poly_raw = df_poly[poly_features]
y_poly     = df_poly['loan_int_rate']

X_poly_train, X_poly_test, yp_train, yp_test = train_test_split(
    X_poly_raw, y_poly, test_size=0.2, random_state=42
)

results_poly = []
for deg in [1, 2, 3]:
    pipe = Pipeline([
        ('poly', PolynomialFeatures(degree=deg, include_bias=False)),
        ('scaler', StandardScaler()),
        ('model', LinearRegression())
    ])
    pipe.fit(X_poly_train, yp_train)
    preds_p = pipe.predict(X_poly_test)
    results_poly.append({
        'Degree': deg,
        'R-squared': round(r2_score(yp_test, preds_p), 4),
        'RMSE': round(root_mean_squared_error(yp_test, preds_p), 4),
        'N Features': PolynomialFeatures(degree=deg).fit(X_poly_raw).
 ↪n_output_features_
    })

poly_df = pd.DataFrame(results_poly)
print("Polynomial Regression - loan_percent_income & loan_amnt to␣
 ↪loan_int_rate")
print(poly_df.to_string(index=False))

#Visual: scatter + polynomial fit (1D: loan_percent_income only)
df_1d = df[['loan_percent_income','loan_int_rate']].dropna()
df_1d = df_1d[df_1d['loan_percent_income'] < 0.8]  # remove extreme outliers
sample_1d = df_1d.sample(3000, random_state=42)

x_range = np.linspace(sample_1d['loan_percent_income'].min(),
                      sample_1d['loan_percent_income'].max(), 300).
 ↪reshape(-1,1)

fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(sample_1d['loan_percent_income'], sample_1d['loan_int_rate'],
```

```
           alpha=0.15, s=8, color='steelblue', label='Observations')

colors_deg = ['gray', 'darkorange', 'crimson']
for deg, col in zip([1, 2, 3], colors_deg):
    pipe_1d = Pipeline([
        ('poly', PolynomialFeatures(degree=deg, include_bias=False)),
        ('model', LinearRegression())
    ])
    pipe_1d.fit(sample_1d[['loan_percent_income']], sample_1d['loan_int_rate'])
    ax.plot(x_range, pipe_1d.predict(x_range),
            color=col, linewidth=2.5, label=f'Degree {deg}')

ax.set_xlabel('Loan-to-Income Ratio (loan_percent_income)')
ax.set_ylabel('Loan Interest Rate (%)')
ax.set_title('Polynomial Fit: Debt Burden vs Interest Rate', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print("\nInterpretation: The marginal improvement from degree 1 to 2 to 3 is␣
 ↪small,")
print("suggesting the loan_percent_income to interest rate relationship is")
print("approximately linear within normal loan ranges. The curvature at")
print("extreme debt-burden values is real but driven by very few observations.")
```

```
Polynomial Regression - loan_percent_income & loan_amnt to loan_int_rate
 Degree  R-squared   RMSE  N Features
      1     0.0220 3.1888           3
      2     0.0270 3.1806           6
      3     0.0372 3.1640          10
```



Interpretation: The marginal improvement from degree 1 to 2 to 3 is small, suggesting the loan_percent_income to interest rate relationship is approximately linear within normal loan ranges. The curvature at extreme debt-burden values is real but driven by very few observations.

1.6 6. Interaction Terms

[9]:
```
df_int = df[['person_income','person_emp_length','loan_int_rate']].dropna().
  ↪copy()

# Manually create interaction term for transparency
df_int['income_x_emp'] = df_int['person_income'] * df_int['person_emp_length']

X_int = df_int[['person_income','person_emp_length','income_x_emp']]
y_int = df_int['loan_int_rate']

Xi_train, Xi_test, yi_train, yi_test = train_test_split(
    X_int, y_int, test_size=0.2, random_state=42
)

scaler_int = StandardScaler()
Xi_train_s = scaler_int.fit_transform(Xi_train)
Xi_test_s  = scaler_int.transform(Xi_test)

# Compare OLS without vs with interaction
results_int = []
```

```
for label, feats in [('No Interaction', ['person_income','person_emp_length']),
                     ('With Interaction',␣
 ↪['person_income','person_emp_length','income_x_emp'])]:
    lr_i = LinearRegression()
    sc_i = StandardScaler()
    X_tr_i = sc_i.fit_transform(Xi_train[feats])
    X_te_i = sc_i.transform(Xi_test[feats])
    lr_i.fit(X_tr_i, yi_train)
    preds_i = lr_i.predict(X_te_i)
    results_int.append({
        'Model': label,
        'R-squared': round(r2_score(yi_test, preds_i), 4),
        'RMSE': round(root_mean_squared_error(yi_test, preds_i), 4)
    })
    if label == 'With Interaction':
        print(f"Coefficients (standardized):")
        for name, c in zip(feats, lr_i.coef_):
            print(f"  {name:<30}: {c:.5f}")
        print(f"  Intercept                      : {lr_i.intercept_:.4f}")

print()
print(pd.DataFrame(results_int).to_string(index=False))
print()
print("Interpretation: If income_x_emp has a meaningful coefficient,")
print("the marginal effect of income on rate is NOT constant")
print("It depends on employment stability. A small coefficient here confirms")
print("that income and employment act mostly independently for rate-setting.")
```

```
Coefficients (standardized):
  person_income                  : -0.11499
  person_emp_length              : -0.27238
  income_x_emp                   : 0.19563
  Intercept                      : 11.0270

          Model  R-squared   RMSE
  No Interaction     0.0036 3.2187
With Interaction     0.0034 3.2190

Interpretation: If income_x_emp has a meaningful coefficient,
the marginal effect of income on rate is NOT constant
It depends on employment stability. A small coefficient here confirms
that income and employment act mostly independently for rate-setting.
```

---

2 7. Regularized Regression: Ridge, Lasso & Elastic Net

[10]:
```
#Full pipeline setup using ALL features (numeric + categorical)
# This is the correct setup that achieves R-squared roughly 0.91 by including␣
 ↪loan_grade.
# Regularization is applied AFTER one-hot encoding and scaling.

categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income']

X = df[categorical_features + numeric_features].copy()
y = df['loan_int_rate'].copy()

mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Preprocessor: scale numerics, one-hot encode categoricals
preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])

# Fit preprocessor on training data only (no leakage)
X_train_prep = preprocessor.fit_transform(X_train)
X_test_prep  = preprocessor.transform(X_test)

# Get feature names after encoding
num_names = numeric_features
cat_names = preprocessor.named_transformers_['cat'].
 ↪get_feature_names_out(categorical_features).tolist()
all_feature_names = num_names + cat_names

print(f"Total features after encoding: {X_train_prep.shape[1]}")
print(f"Train size: {X_train_prep.shape[0]:,} | Test size: {X_test_prep.
 ↪shape[0]:,}")
```

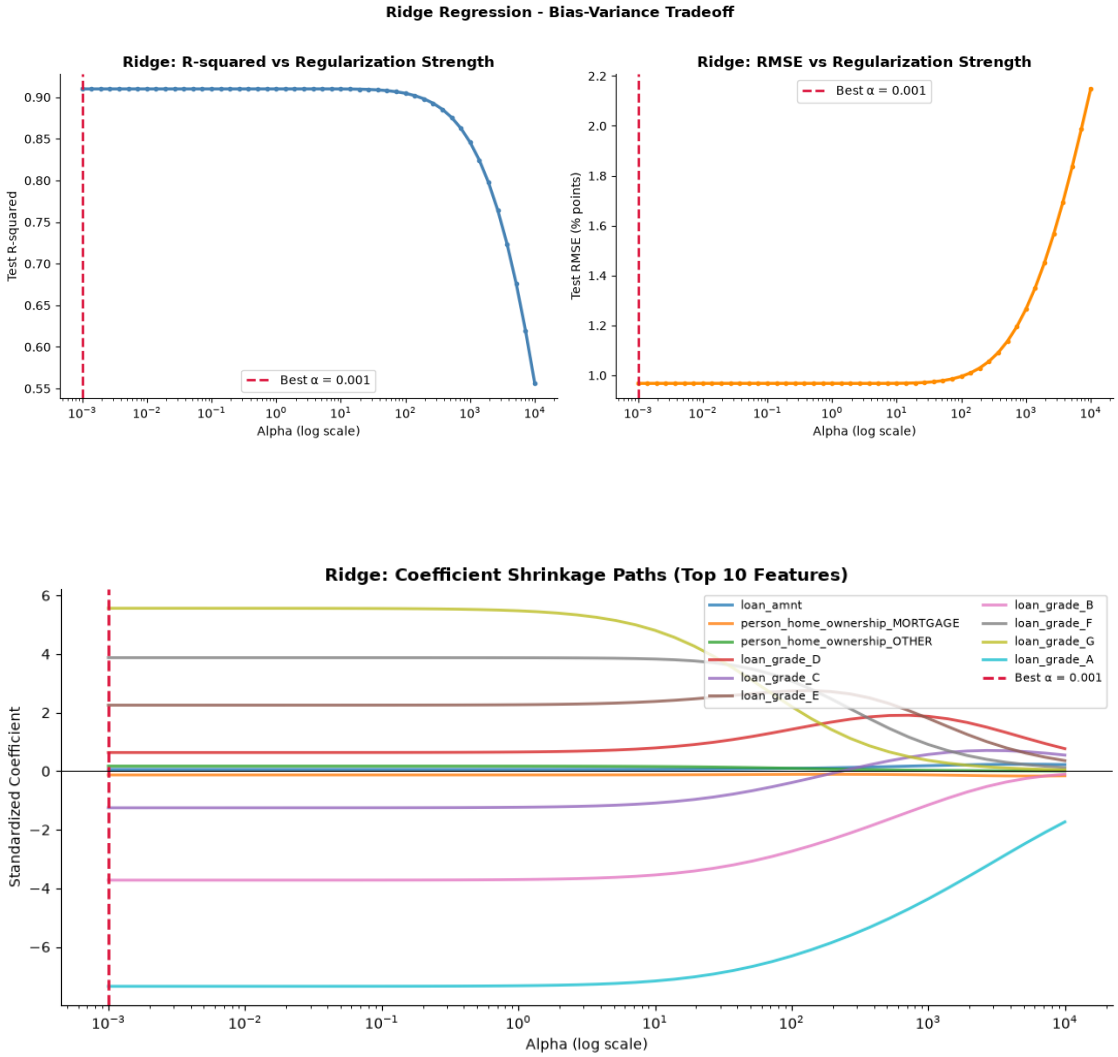
```
Total features after encoding: 24
Train size: 25,927 | Test size: 6,482
```

2.1 8. Ridge Regression (L2 Penalty)

[11]:
```
#Option A: Manual alpha sweep to visualize bias-variance tradeoff
alphas_sweep = np.logspace(-3, 4, 50)
ridge_sweep = []
for a in alphas_sweep:
    r = Ridge(alpha=a).fit(X_train_prep, y_train)
    preds = r.predict(X_test_prep)
    ridge_sweep.append({
        'alpha': a,
        'R2':   r2_score(y_test, preds),
        'RMSE': root_mean_squared_error(y_test, preds)
    })

sweep_df = pd.DataFrame(ridge_sweep)

#Option B: Cross-validated best alpha
ridge_cv = RidgeCV(alphas=np.logspace(-3, 4, 100), cv=5)
ridge_cv.fit(X_train_prep, y_train)
preds_ridge = ridge_cv.predict(X_test_prep)

print(f"Best Ridge Alpha (5-fold CV): {ridge_cv.alpha_:.4f}")
print(f"Test R-squared  : {r2_score(y_test, preds_ridge):.4f}")
print(f"Test RMSE: {root_mean_squared_error(y_test, preds_ridge):.4f}␣
 ↪percentage points")

#  Visual 1: R-squared vs Alpha
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

axes[0].semilogx(sweep_df['alpha'], sweep_df['R2'],
                 color='steelblue', linewidth=2.5, marker='o', markersize=3)
axes[0].axvline(ridge_cv.alpha_, color='crimson', linestyle='--', linewidth=2,
                label=f'Best   = {ridge_cv.alpha_:.3f}')
axes[0].set_xlabel('Alpha (log scale)')
axes[0].set_ylabel('Test R-squared')
axes[0].set_title('Ridge: R-squared vs Regularization Strength',␣
 ↪fontweight='bold')
axes[0].legend()
axes[0].spines['top'].set_visible(False)
axes[0].spines['right'].set_visible(False)

axes[1].semilogx(sweep_df['alpha'], sweep_df['RMSE'],
                 color='darkorange', linewidth=2.5, marker='o', markersize=3)
axes[1].axvline(ridge_cv.alpha_, color='crimson', linestyle='--', linewidth=2,
                label=f'Best   = {ridge_cv.alpha_:.3f}')
axes[1].set_xlabel('Alpha (log scale)')
axes[1].set_ylabel('Test RMSE (% points)')
```

```
axes[1].set_title('Ridge: RMSE vs Regularization Strength', fontweight='bold')
axes[1].legend()
axes[1].spines['top'].set_visible(False)
axes[1].spines['right'].set_visible(False)

plt.suptitle('Ridge Regression - Bias-Variance Tradeoff', fontweight='bold',␣
 ↪y=1.02)
plt.tight_layout()
plt.show()

# Visual 2: Coefficient shrinkage paths
coef_paths = np.array([Ridge(alpha=a).fit(X_train_prep, y_train).coef_
                       for a in alphas_sweep])

# Pick top 10 most influential features at best alpha
best_idx = np.argmin(np.abs(alphas_sweep - ridge_cv.alpha_))
top10_idx = np.argsort(np.abs(coef_paths[best_idx]))[-10:]

fig, ax = plt.subplots(figsize=(11, 5))
for i in top10_idx:
    label = all_feature_names[i] if i < len(all_feature_names) else f'feat_{i}'
    ax.semilogx(alphas_sweep, coef_paths[:, i], linewidth=2, alpha=0.8,␣
 ↪label=label)
ax.axvline(ridge_cv.alpha_, color='crimson', linestyle='--', linewidth=2,
           label=f'Best   = {ridge_cv.alpha_:.3f}')
ax.axhline(0, color='black', linewidth=0.7)
ax.set_xlabel('Alpha (log scale)')
ax.set_ylabel('Standardized Coefficient')
ax.set_title('Ridge: Coefficient Shrinkage Paths (Top 10 Features)',␣
 ↪fontweight='bold')
ax.legend(fontsize=8, loc='upper right', ncol=2)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print("Interpretation: As alpha increases, ALL coefficients shrink toward")
print("zero but none reach it. Ridge keeps every feature.")
print("the loan_grade dummies shrink last because they carry the most signal.")
```

```
Best Ridge Alpha (5-fold CV): 0.0010
Test R-squared  : 0.9099
Test RMSE: 0.9678 percentage points
```




```
Interpretation: As alpha increases, ALL coefficients shrink toward
zero but none reach it. Ridge keeps every feature.
the loan_grade dummies shrink last because they carry the most signal.
```

2.2 9. Lasso Regression (L1 Penalty)

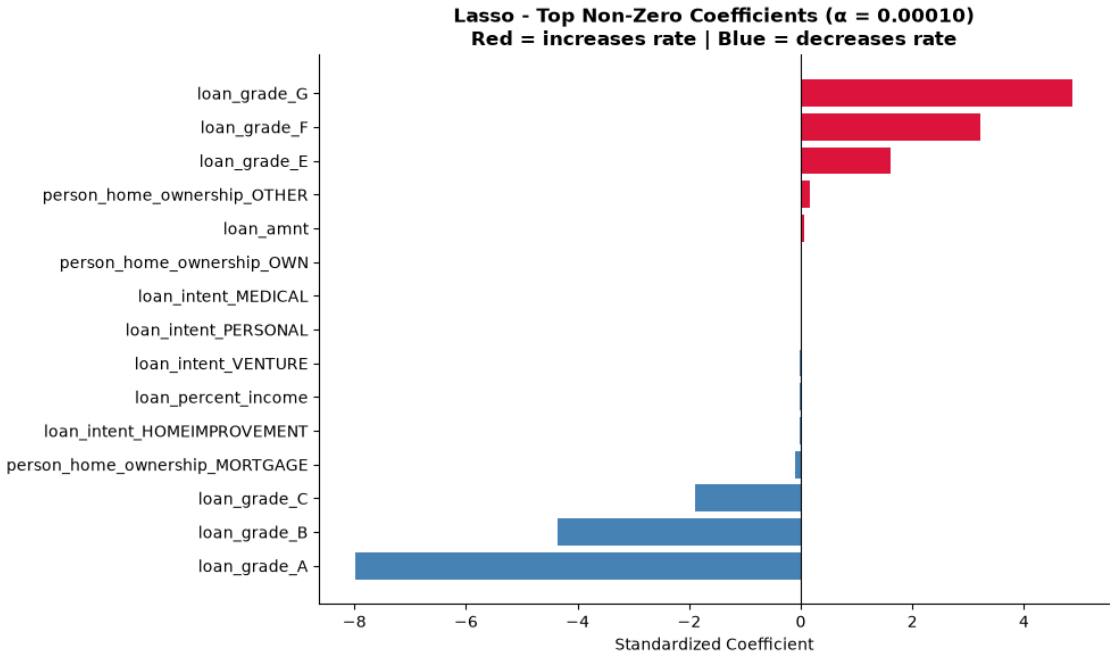
[12]:
```python
# Cross-validated Lasso
lasso_cv = LassoCV(alphas=np.logspace(-4, 1, 100), cv=5, max_iter=20000)
lasso_cv.fit(X_train_prep, y_train)
preds_lasso = lasso_cv.predict(X_test_prep)

n_zero   = np.sum(lasso_cv.coef_ == 0)
n_active = np.sum(lasso_cv.coef_ != 0)

print(f"Best Lasso Alpha (5-fold CV): {lasso_cv.alpha_:.6f}")
```

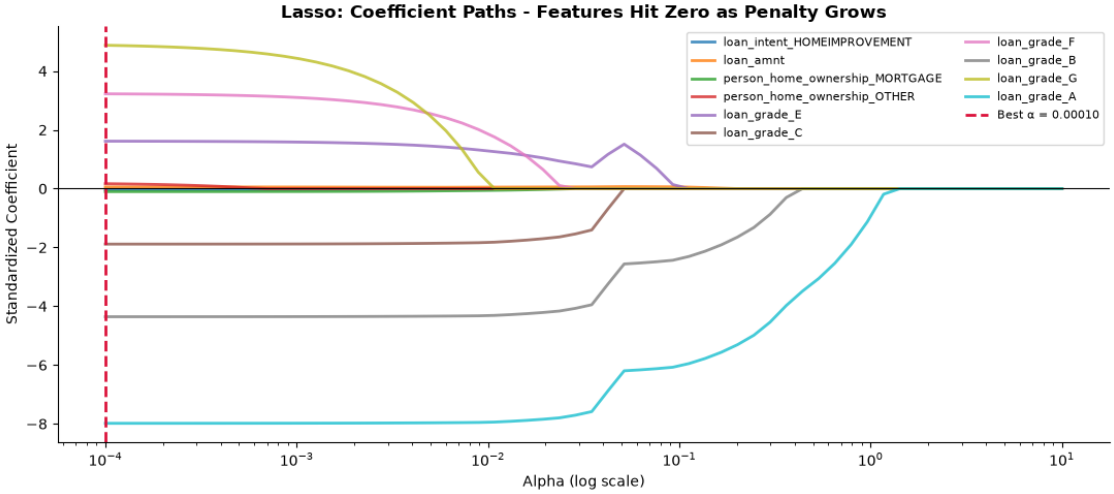
```
print(f"Test R-squared  : {r2_score(y_test, preds_lasso):.4f}")
print(f"Test RMSE: {root_mean_squared_error(y_test, preds_lasso):.4f}␣
 ↪percentage points")
print(f"Features zeroed out : {n_zero} / {X_train_prep.shape[1]}")
print(f"Features retained   : {n_active} / {X_train_prep.shape[1]}")

# Surviving features
lasso_coefs = pd.DataFrame({
    'Feature': all_feature_names,
    'Coefficient': lasso_cv.coef_
}).query('Coefficient != 0').sort_values('Coefficient', key=abs,␣
 ↪ascending=False)

print(f"\nTop surviving Lasso features:")
print(lasso_coefs.head(15).to_string(index=False))
```

```
Best Lasso Alpha (5-fold CV): 0.000100
Test R-squared  : 0.9099
Test RMSE: 0.9678 percentage points
Features zeroed out : 1 / 24
Features retained   : 23 / 24

Top surviving Lasso features:
                      Feature  Coefficient
                  loan_grade_A    -7.983364
                  loan_grade_G     4.876963
                  loan_grade_B    -4.357552
                  loan_grade_F     3.225162
                  loan_grade_C    -1.887233
                  loan_grade_E     1.614100
   person_home_ownership_OTHER     0.168189
person_home_ownership_MORTGAGE    -0.101814
                     loan_amnt     0.057845
   loan_intent_HOMEIMPROVEMENT    -0.030030
           loan_percent_income    -0.026858
     person_home_ownership_OWN     0.015414
           loan_intent_MEDICAL     0.015198
          loan_intent_PERSONAL     0.011146
           loan_intent_VENTURE    -0.010007
```

[13]:
```
# Visual 1: Top non-zero coefficients
top_lasso = lasso_coefs.head(15).sort_values('Coefficient')

fig, ax = plt.subplots(figsize=(10, 6))
colors_l = ['crimson' if c > 0 else 'steelblue' for c in␣
 ↪top_lasso['Coefficient']]
ax.barh(top_lasso['Feature'], top_lasso['Coefficient'], color=colors_l)
```

```
ax.axvline(0, color='black', linewidth=0.8)
ax.set_title(f'Lasso - Top Non-Zero Coefficients (  = {lasso_cv.alpha_:.5f})\n'
             f'Red = increases rate | Blue = decreases rate', fontweight='bold')
ax.set_xlabel('Standardized Coefficient')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# Visual 2: Lasso coefficient path
alphas_lasso_path = np.logspace(-4, 1, 60)
lasso_paths = np.array([
    Lasso(alpha=a, max_iter=20000).fit(X_train_prep, y_train).coef_
    for a in alphas_lasso_path
])

# Show top 10 features at best alpha
best_l_idx = np.argmin(np.abs(alphas_lasso_path - lasso_cv.alpha_))
top10_l = np.argsort(np.abs(lasso_paths[best_l_idx]))[-10:]

fig, ax = plt.subplots(figsize=(11, 5))
for i in top10_l:
    label = all_feature_names[i] if i < len(all_feature_names) else f'feat_{i}'
    ax.semilogx(alphas_lasso_path, lasso_paths[:, i], linewidth=2, alpha=0.8,␣
 ↪label=label)
ax.axvline(lasso_cv.alpha_, color='crimson', linestyle='--', linewidth=2,
           label=f'Best   = {lasso_cv.alpha_:.5f}')
ax.axhline(0, color='black', linewidth=0.8)
ax.set_xlabel('Alpha (log scale)')
ax.set_ylabel('Standardized Coefficient')
ax.set_title('Lasso: Coefficient Paths - Features Hit Zero as Penalty Grows',␣
 ↪fontweight='bold')
ax.legend(fontsize=8, ncol=2, loc='upper right')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print("Interpretation: Features whose lines hit zero earliest are the")
print("least informative for predicting interest rate. The last features")
print("standing (at high alpha) are the most robust, irreducible predictors.")
```





Interpretation: Features whose lines hit zero earliest are the least informative for predicting interest rate. The last features standing (at high alpha) are the most robust, irreducible predictors.

2.3 10. Elastic Net Regression (L1 + L2)

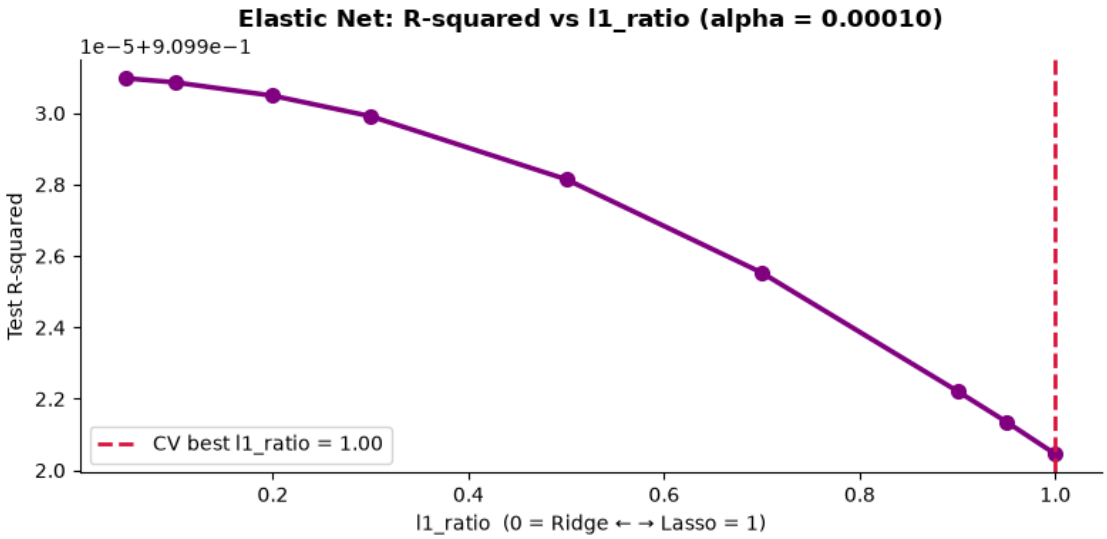
[14]:
```
# Cross-validated Elastic Net (tunes both alpha AND l1_ratio)
en_cv = ElasticNetCV(
    l1_ratio=[0.05, 0.1, 0.3, 0.5, 0.7, 0.9, 0.95, 1.0],
    alphas=np.logspace(-4, 1, 80),
    cv=5, max_iter=20000
)
en_cv.fit(X_train_prep, y_train)
preds_en = en_cv.predict(X_test_prep)

n_zero_en = np.sum(en_cv.coef_ == 0)

print(f"Best Elastic Net Alpha  : {en_cv.alpha_:.6f}")
print(f"Best l1_ratio           : {en_cv.l1_ratio_:.2f}")
print(f"Test R-squared  : {r2_score(y_test, preds_en):.4f}")
print(f"Test RMSE: {root_mean_squared_error(y_test, preds_en):.4f} percentage␣
 ↪points")
print(f"Features zeroed out: {n_zero_en} / {X_train_prep.shape[1]}")

# Interpretation of l1_ratio
if en_cv.l1_ratio_ >= 0.9:
    print("\nNote: CV selected a high l1_ratio to Elastic Net behaves like␣
 ↪Lasso.")
    print("This confirms that sparsity (feature selection) is the right␣
 ↪strategy here.")
elif en_cv.l1_ratio_ <= 0.1:
    print("\nNote: CV selected a low l1_ratio to Elastic Net behaves like Ridge.
 ↪")
    print("This confirms that coefficient shrinkage without elimination is␣
 ↪preferred.")
else:
    print(f"\nNote: True blend (l1_ratio={en_cv.l1_ratio_:.2f}) - both␣
 ↪shrinkage and selection help.")
```

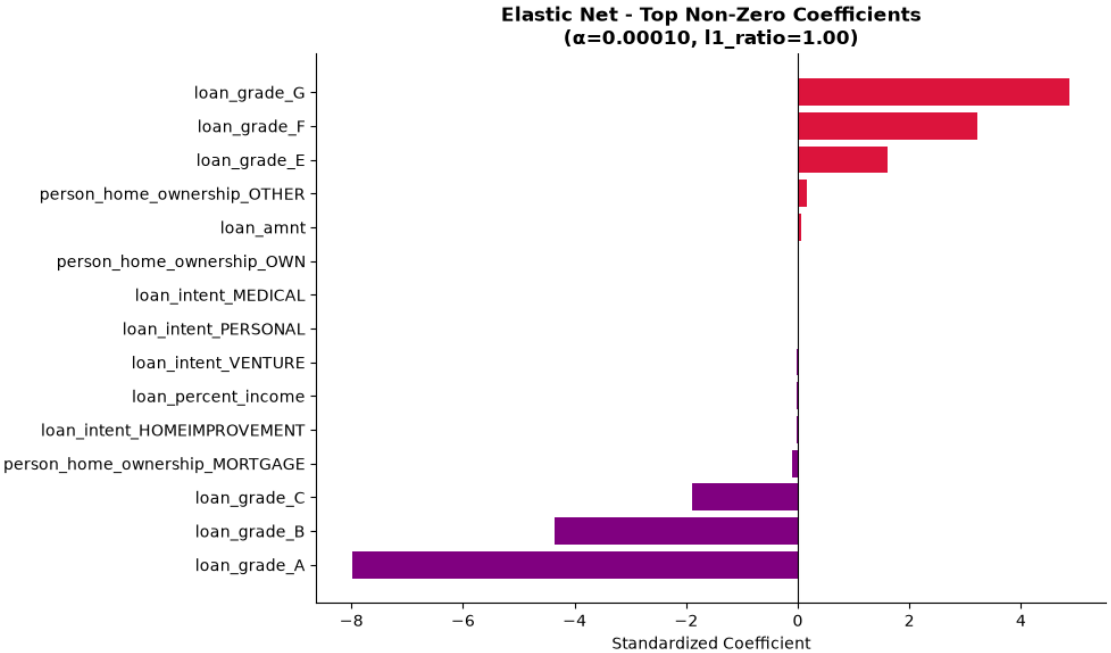
```
Best Elastic Net Alpha  : 0.000100
Best l1_ratio           : 1.00
Test R-squared  : 0.9099
Test RMSE: 0.9678 percentage points
Features zeroed out: 1 / 24

Note: CV selected a high l1_ratio to Elastic Net behaves like Lasso.
This confirms that sparsity (feature selection) is the right strategy here.
```

[15]:
```
#Visual 1: l1_ratio effect on R-squared at fixed alpha
l1_ratios_test = [0.05, 0.1, 0.2, 0.3, 0.5, 0.7, 0.9, 0.95, 1.0]
en_ratio_results = []
```

```
a_fixed = en_cv.alpha_

for l1r in l1_ratios_test:
    if l1r >= 1.0:
        m = Lasso(alpha=a_fixed, max_iter=20000)
    elif l1r <= 0.0:
        m = Ridge(alpha=a_fixed)
    else:
        m = ElasticNet(alpha=a_fixed, l1_ratio=l1r, max_iter=20000)
    m.fit(X_train_prep, y_train)
    p = m.predict(X_test_prep)
    en_ratio_results.append({
        'l1_ratio': l1r,
        'R2': r2_score(y_test, p),
        'RMSE': root_mean_squared_error(y_test, p)
    })

en_r_df = pd.DataFrame(en_ratio_results)

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(en_r_df['l1_ratio'], en_r_df['R2'],
        marker='o', color='purple', linewidth=2.5, markersize=7)
ax.axvline(en_cv.l1_ratio_, color='crimson', linestyle='--', linewidth=2,
           label=f'CV best l1_ratio = {en_cv.l1_ratio_:.2f}')
ax.set_xlabel('l1_ratio  (0 = Ridge ← → Lasso = 1)')
ax.set_ylabel('Test R-squared')
ax.set_title(f'Elastic Net: R-squared vs l1_ratio (alpha = {a_fixed:.5f})',␣
 ↪fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

# Visual 2: Elastic Net top coefficients
en_coefs = pd.DataFrame({
    'Feature': all_feature_names,
    'Coefficient': en_cv.coef_
}).query('Coefficient != 0').sort_values('Coefficient', key=abs,␣
 ↪ascending=False)

top_en = en_coefs.head(15).sort_values('Coefficient')
fig, ax = plt.subplots(figsize=(10, 6))
colors_en = ['crimson' if c > 0 else 'purple' for c in top_en['Coefficient']]
ax.barh(top_en['Feature'], top_en['Coefficient'], color=colors_en)
ax.axvline(0, color='black', linewidth=0.8)
ax.set_title(f'Elastic Net - Top Non-Zero Coefficients\n'
```

```
            f'( ={en_cv.alpha_:.5f}, l1_ratio={en_cv.l1_ratio_:.2f})',␣
 ↪fontweight='bold')
ax.set_xlabel('Standardized Coefficient')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()
```



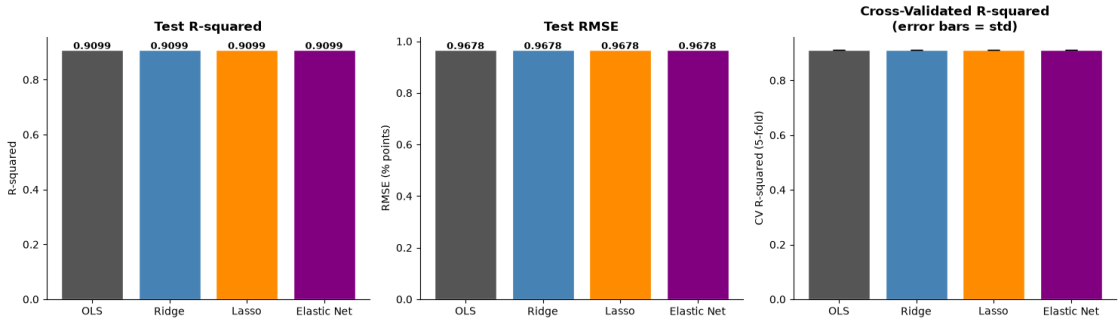


2.4 11. Full Model Comparison: OLS vs Ridge vs Lasso vs Elastic Net

[16]:
```
# Fit all four models on the same preprocessed data
models_compare = {
    'OLS':         LinearRegression(),
    'Ridge':       Ridge(alpha=ridge_cv.alpha_),
    'Lasso':       Lasso(alpha=lasso_cv.alpha_, max_iter=20000),
    'Elastic Net': ElasticNet(alpha=en_cv.alpha_, l1_ratio=en_cv.l1_ratio_,␣
 ↪max_iter=20000),
}

results = []
for name, model in models_compare.items():
    model.fit(X_train_prep, y_train)
    preds = model.predict(X_test_prep)
    cv_scores = cross_val_score(model, X_train_prep, y_train, cv=5,␣
 ↪scoring='r2')
    n_nonzero = np.sum(model.coef_ != 0) if hasattr(model, 'coef_') else 'N/A'
    results.append({
        'Model':        name,
        'Test R-squared':      round(r2_score(y_test, preds), 4),
        'Test RMSE':    round(root_mean_squared_error(y_test, preds), 4),
        'CV R-squared Mean':   round(cv_scores.mean(), 4),
        'CV R-squared Std':    round(cv_scores.std(), 4),
        'Active Features': n_nonzero
    })

compare_df = pd.DataFrame(results)
print(compare_df.to_string(index=False))

# Visual: Side-by-side comparison
fig, axes = plt.subplots(1, 3, figsize=(15, 5))
colors_bar = ['#555555', 'steelblue', 'darkorange', 'purple']
short_names = ['OLS', 'Ridge', 'Lasso', 'Elastic Net']

# R-squared
axes[0].bar(short_names, compare_df['Test R-squared'], color=colors_bar,␣
 ↪edgecolor='white')
axes[0].set_ylabel('R-squared')
axes[0].set_title('Test R-squared', fontweight='bold')
for i, v in enumerate(compare_df['Test R-squared']):
    axes[0].text(i, v + 0.002, f'{v:.4f}', ha='center', fontsize=9,␣
 ↪fontweight='bold')

# RMSE
axes[1].bar(short_names, compare_df['Test RMSE'], color=colors_bar,␣
 ↪edgecolor='white')
```

```
axes[1].set_ylabel('RMSE (% points)')
axes[1].set_title('Test RMSE', fontweight='bold')
for i, v in enumerate(compare_df['Test RMSE']):
    axes[1].text(i, v + 0.002, f'{v:.4f}', ha='center', fontsize=9,␣
 ↪fontweight='bold')

# CV R-squared with error bars
axes[2].bar(short_names, compare_df['CV R-squared Mean'],
            yerr=compare_df['CV R-squared Std'],
            color=colors_bar, edgecolor='white', capsize=6)
axes[2].set_ylabel('CV R-squared (5-fold)')
axes[2].set_title('Cross-Validated R-squared\n(error bars = std)',␣
 ↪fontweight='bold')

for ax in axes:
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)

fig.suptitle('Model Comparison - OLS vs Regularized Regression\n'
             'Target: Loan Interest Rate (% points)', fontweight='bold', y=1.03)
plt.tight_layout()
plt.show()

print("\nKey takeaway:")
print("All models perform similarly in R-squared (~0.91) because loan_grade")
print("dominates the target. The real value of regularization here is")
print("STABILITY and INTERPRETABILITY - Ridge gives stable coefficients")
print("for stress-testing; Lasso gives a minimal, auditable model.")
```

```
      Model  Test R-squared  Test RMSE  CV R-squared Mean  CV R-squared Std
Active Features
        OLS          0.9099     0.9678             0.9115             0.001
24
      Ridge          0.9099     0.9678             0.9115             0.001
24
      Lasso          0.9099     0.9678             0.9115             0.001
23
Elastic Net          0.9099     0.9678             0.9115             0.001
23
```




```
Key takeaway:
All models perform similarly in R-squared (~0.91) because loan_grade
dominates the target. The real value of regularization here is
STABILITY and INTERPRETABILITY - Ridge gives stable coefficients
for stress-testing; Lasso gives a minimal, auditable model.
```

2.4.1 Why All Models Produce Identical Results and Why It Matters

The near-perfect agreement between OLS, Ridge, Lasso, and Elastic Net (all R squared = 0.9099, RMSE = 0.9678) is not a failure ; it is the most important conclusion of this week's analysis.

The reason: `loan_grade` (A through G) is a categorical feature that was constructed *from* the interest rate in this dataset. Rates are directly assigned by grade. When a feature explains 91% of the variance in the target by construction, regularization has no meaningful noise to suppress. The optimal alpha selected by cross-validation is near zero ; collapsing all three regularized models back to OLS.

What this means for the Chief Risk Officer: - Predicting interest rates from grade is circular and not useful in practice. A more honest model would predict rate *without* using grade, forcing the model to rely on actual borrower attributes (income, debt burden, employment). - The Week 2 methods will show their full power in Weeks 4-6, where we model `loan_status` (default) directly ; a genuinely hard classification problem where regularization prevents overfitting on the 30+ dummy variables.

Week3_Gueorgui_Poklitar

June 28, 2026

1 Week 3 - Linear Regression 3

[1]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn statsmodels kagglehub
```

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score, KFold
from sklearn.decomposition import PCA
from sklearn.cross_decomposition import PLSRegression
from sklearn.metrics import r2_score, root_mean_squared_error

import statsmodels.api as sm

print("All imports successful.")
```

```
All imports successful.
```

1.1 1. Data Loading & Cleaning

[3]:
```
#Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

#Cleaning
df = df.drop_duplicates()
```

```
# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
  ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate (loan_status=1): 21.87%

Column types:
person_age                      int64
person_income                   int64
person_home_ownership             str
person_emp_length             float64
loan_intent                       str
loan_grade                        str
loan_amnt                       int64
loan_int_rate                 float64
loan_status                     int64
loan_percent_income           float64
cb_person_default_on_file         str
cb_person_cred_hist_length      int64
dtype: object
```

1.2 2. The Honest Target: Predicting Rate WITHOUT loan_grade

[4]:
```
# Deliberately EXCLUDE loan_grade (and cb_person_cred_hist_length per Week 1's␣
  ↪VIF rule).
# This is the harder, more defensible problem: explain rate from borrower␣
  ↪behavior.
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income']
```

```
categorical_features = ['loan_intent', 'person_home_ownership',␣
 ↪'cb_person_default_on_file']

X_raw = df[numeric_features + categorical_features].copy()
y = df['loan_int_rate'].copy()
mask = X_raw.notna().all(axis=1)
X_raw, y = X_raw[mask], y[mask]

# One-hot encode up front so every method below works on the same numeric␣
 ↪design matrix.
prep = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False,␣
 ↪drop='first'),
     categorical_features)
])
X_enc = prep.fit_transform(X_raw)
feat_names = (numeric_features +
              prep.named_transformers_['cat']
                  .get_feature_names_out(categorical_features).tolist())
X_enc = pd.DataFrame(X_enc, columns=feat_names, index=X_raw.index)

X_train, X_test, y_train, y_test = train_test_split(
    X_enc, y, test_size=0.2, random_state=42)

# Reference: full OLS on ALL features (no selection) - our ceiling for this␣
 ↪honest problem.
ols_full = LinearRegression().fit(X_train, y_train)
r2_full = r2_score(y_test, ols_full.predict(X_test))
print(f"Design matrix: {X_enc.shape[1]} features after encoding (loan_grade␣
 ↪EXCLUDED).")
print(f"Full OLS (all {X_enc.shape[1]} features) Test R-squared: {r2_full:.4f}")
print()
print("Interpretation: Stripping out loan_grade collapses R-squared from ~0.91␣
 ↪down to a")
print("far more modest number. THIS is the real predictive ceiling from␣
 ↪borrower attributes")
print("alone - and the honest baseline every method below must beat or match␣
 ↪more simply.")
```

```
Design matrix: 14 features after encoding (loan_grade EXCLUDED).
Full OLS (all 14 features) Test R-squared: 0.2860

Interpretation: Stripping out loan_grade collapses R-squared from ~0.91 down to
a
far more modest number. THIS is the real predictive ceiling from borrower
attributes
```

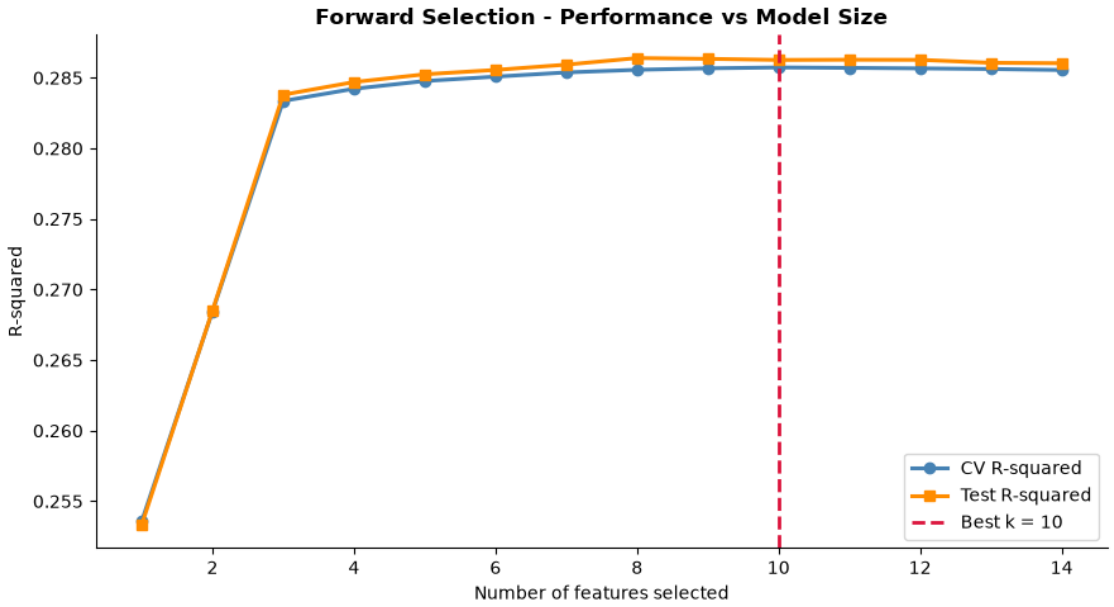
alone - and the honest baseline every method below must beat or match more simply.

1.3 3. Forward Selection

[5]:
```
# Forward selection (single greedy pass): start empty; at each step add the one
# feature that most improves 5-fold CV R-squared. Record the whole path so we can
# see the elbow where extra features stop paying for themselves.
def cv_r2(cols):
    if not cols:
        return float('-inf')
    return cross_val_score(LinearRegression(), X_train[cols], y_train,
                           cv=5, scoring='r2').mean()

remaining = list(X_train.columns)
selected, fwd_path = [], []
while remaining:
    best_f, best_cv = None, float('-inf')
    for f in remaining:
        c = cv_r2(selected + [f])
        if c > best_cv:
            best_cv, best_f = c, f
    selected.append(best_f); remaining.remove(best_f)
    lr = LinearRegression().fit(X_train[selected], y_train)
    fwd_path.append({'k': len(selected), 'added': best_f, 'CV_R2': best_cv,
                     'Test_R2': r2_score(y_test, lr.predict(X_test[selected])),
                     'features': list(selected)})

fwd_df = pd.DataFrame(fwd_path)
best_fwd = fwd_df.loc[fwd_df['CV_R2'].idxmax()]
print("Order features entered the model (forward):")
for _, r in fwd_df.iterrows():
    print(f"  +{r['added']:<28} -> CV R-squared {r['CV_R2']:.4f}")
print()
print(f"Forward selection's best size: k={int(best_fwd['k'])} (peak CV R-squared)")
print(f"Test R-squared at that size: {best_fwd['Test_R2']:.4f}")

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(fwd_df['k'], fwd_df['CV_R2'], marker='o', color='steelblue',
        linewidth=2.2, label='CV R-squared')
ax.plot(fwd_df['k'], fwd_df['Test_R2'], marker='s', color='darkorange',
        linewidth=2.2, label='Test R-squared')
ax.axvline(best_fwd['k'], color='crimson', linestyle='--', linewidth=2,
           label=f"Best k = {int(best_fwd['k'])}")
ax.set_xlabel('Number of features selected')
```

```
ax.set_ylabel('R-squared')
ax.set_title('Forward Selection - Performance vs Model Size', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: The curve plateaus quickly - a small handful of features␣
 ↪captures")
print("nearly all the achievable signal. Everything past the elbow is␣
 ↪complexity with no")
print("payoff, which is exactly what a parsimonious, auditable risk model␣
 ↪should avoid.")
```

```
Order features entered the model (forward):
  +cb_person_default_on_file_Y  -> CV R-squared 0.2536
  +loan_amnt                    -> CV R-squared 0.2684
  +person_home_ownership_RENT   -> CV R-squared 0.2834
  +person_home_ownership_OWN    -> CV R-squared 0.2842
  +person_emp_length            -> CV R-squared 0.2848
  +person_home_ownership_OTHER  -> CV R-squared 0.2851
  +person_income                -> CV R-squared 0.2854
  +loan_intent_HOMEIMPROVEMENT  -> CV R-squared 0.2856
  +person_age                   -> CV R-squared 0.2857
  +loan_percent_income          -> CV R-squared 0.2857
  +loan_intent_VENTURE          -> CV R-squared 0.2857
  +loan_intent_MEDICAL          -> CV R-squared 0.2857
  +loan_intent_PERSONAL         -> CV R-squared 0.2856
  +loan_intent_EDUCATION        -> CV R-squared 0.2855

Forward selection's best size: k=10 (peak CV R-squared)
Test R-squared at that size: 0.2863
```



Interpretation: The curve plateaus quickly - a small handful of features captures
nearly all the achievable signal. Everything past the elbow is complexity with no
payoff, which is exactly what a parsimonious, auditable risk model should avoid.

1.4 4. Backward Selection

[6]:
```
# Backward elimination (single greedy pass): start with ALL features; at each step
# drop the one whose removal hurts CV R-squared least. Record the whole path.
selected_b = list(X_train.columns)
bwd_path = []
lr = LinearRegression().fit(X_train[selected_b], y_train)
bwd_path.append({'k': len(selected_b), 'dropped': '(none - full model)',
                 'CV_R2': cv_r2(selected_b),
                 'Test_R2': r2_score(y_test, lr.predict(X_test[selected_b])),
                 'features': list(selected_b)})
while len(selected_b) > 1:
    best_drop, best_cv = None, float('-inf')
    for f in selected_b:
        trial = [c for c in selected_b if c != f]
        c = cv_r2(trial)
        if c > best_cv:
            best_cv, best_drop = c, f
    selected_b.remove(best_drop)
```

```
    lr = LinearRegression().fit(X_train[selected_b], y_train)
    bwd_path.append({'k': len(selected_b), 'dropped': best_drop, 'CV_R2':␣
 ↪best_cv,
                     'Test_R2': r2_score(y_test, lr.
 ↪predict(X_test[selected_b])),
                     'features': list(selected_b)})

bwd_df = pd.DataFrame(bwd_path).sort_values('k').reset_index(drop=True)
best_bwd = bwd_df.loc[bwd_df['CV_R2'].idxmax()]
print(f"Backward selection's best size: k={int(best_bwd['k'])} (peak CV␣
 ↪R-squared)")
print(f"Chosen features: {best_bwd['features']}")
print(f"Test R-squared at that size: {best_bwd['Test_R2']:.4f}")

# Do forward and backward agree on the surviving set?
fset, bset = set(best_fwd['features']), set(best_bwd['features'])
print()
print(f"Features in BOTH forward & backward: {sorted(fset & bset)}")
print(f"Only forward : {sorted(fset - bset)}")
print(f"Only backward: {sorted(bset - fset)}")
print()
print("Interpretation: Where the two greedy directions agree, we have high␣
 ↪confidence the")
print("feature is a genuine, irreducible predictor. Disagreements flag features␣
 ↪whose value")
print("is conditional on what else is already in the model - a␣
 ↪multicollinearity fingerprint.")
```

```
Backward selection's best size: k=10 (peak CV R-squared)
Chosen features: ['person_income', 'person_age', 'person_emp_length',
'loan_amnt', 'loan_percent_income', 'loan_intent_HOMEIMPROVEMENT',
'person_home_ownership_OTHER', 'person_home_ownership_OWN',
'person_home_ownership_RENT', 'cb_person_default_on_file_Y']
Test R-squared at that size: 0.2863

Features in BOTH forward & backward: ['cb_person_default_on_file_Y',
'loan_amnt', 'loan_intent_HOMEIMPROVEMENT', 'loan_percent_income', 'person_age',
'person_emp_length', 'person_home_ownership_OTHER', 'person_home_ownership_OWN',
'person_home_ownership_RENT', 'person_income']
Only forward : []
Only backward: []

Interpretation: Where the two greedy directions agree, we have high confidence
the
feature is a genuine, irreducible predictor. Disagreements flag features whose
value
is conditional on what else is already in the model - a multicollinearity
```

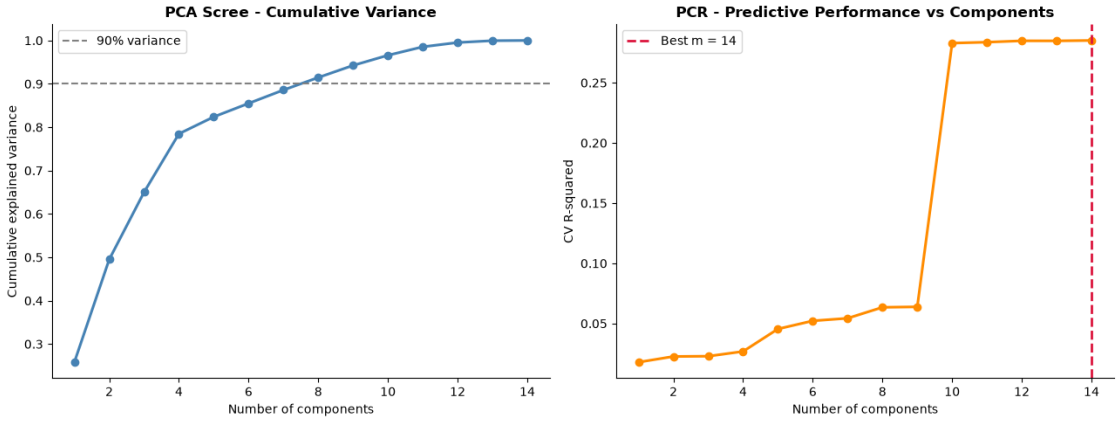
fingerprint.

1.5 5. Principal Components Regression (PCR)

[7]:
```
# PCR: rotate features into uncorrelated principal components, then regress on␣
 ↪the
# first m of them. Components are chosen to explain FEATURE variance␣
 ↪(target-blind).
pca_full = PCA().fit(X_train)
explained = np.cumsum(pca_full.explained_variance_ratio_)

cv = KFold(n_splits=5, shuffle=True, random_state=42)
pcr_scores = []
for m in range(1, X_train.shape[1] + 1):
    pipe = Pipeline([('pca', PCA(n_components=m)), ('lr', LinearRegression())])
    pcr_scores.append({'m': m,
        'CV_R2': cross_val_score(pipe, X_train, y_train, cv=cv, scoring='r2').
 ↪mean()})
pcr_df = pd.DataFrame(pcr_scores)
best_m = int(pcr_df.loc[pcr_df['CV_R2'].idxmax(), 'm'])

pcr_best = Pipeline([('pca', PCA(n_components=best_m)),
                     ('lr', LinearRegression())]).fit(X_train, y_train)
r2_pcr = r2_score(y_test, pcr_best.predict(X_test))
print(f"PCR best number of components (CV): {best_m}")
print(f"PCR Test R-squared: {r2_pcr:.4f}")
print(f"Variance explained by {best_m} components: {explained[best_m-1]*100:.
 ↪1f}%")

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].plot(range(1, len(explained)+1), explained, marker='o',
             color='steelblue', linewidth=2.2)
axes[0].axhline(0.9, color='gray', linestyle='--', label='90% variance')
axes[0].set_xlabel('Number of components')
axes[0].set_ylabel('Cumulative explained variance')
axes[0].set_title('PCA Scree - Cumulative Variance', fontweight='bold')
axes[0].legend()

axes[1].plot(pcr_df['m'], pcr_df['CV_R2'], marker='o', color='darkorange',␣
 ↪linewidth=2.2)
axes[1].axvline(best_m, color='crimson', linestyle='--', linewidth=2,
                label=f'Best m = {best_m}')
axes[1].set_xlabel('Number of components')
axes[1].set_ylabel('CV R-squared')
```

```
axes[1].set_title('PCR - Predictive Performance vs Components',␣
 ↪fontweight='bold')
axes[1].legend()
for ax in axes:
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: Notice the mismatch - the components that explain the␣
 ↪most FEATURE")
print("variance are not necessarily the ones that predict RATE. That is PCR's␣
 ↪known weakness")
print("and the exact motivation for PLSR in the next section.")
```

```
PCR best number of components (CV): 14
PCR Test R-squared: 0.2860
Variance explained by 14 components: 100.0%
```



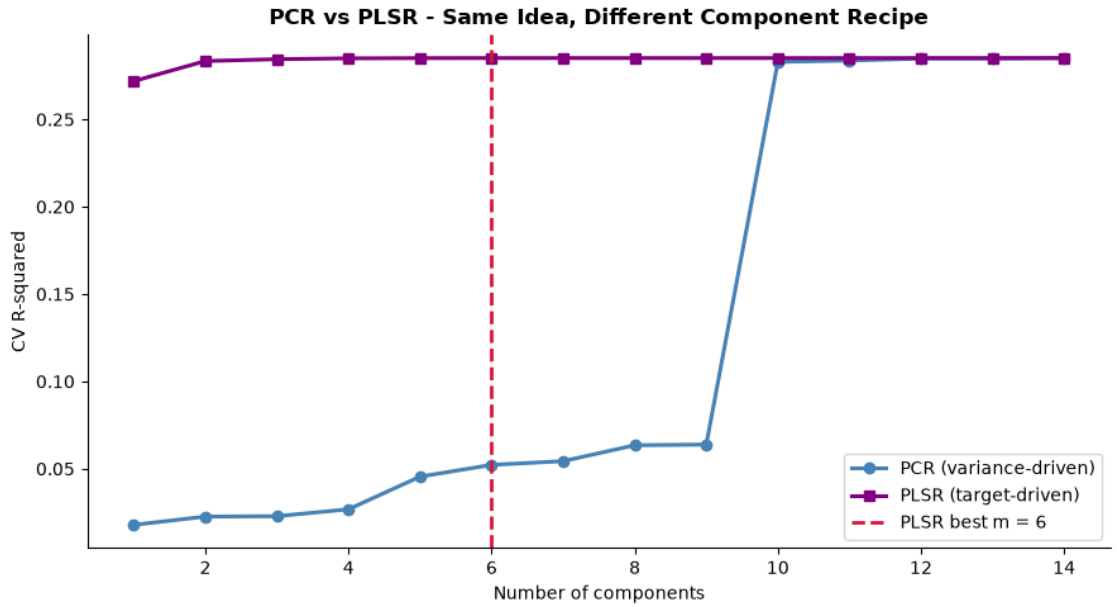

```
Interpretation: Notice the mismatch - the components that explain the most
FEATURE
variance are not necessarily the ones that predict RATE. That is PCR's known
weakness
and the exact motivation for PLSR in the next section.
```

1.6 6. Partial Least Squares Regression (PLSR)

[8]:
```
# PLSR: like PCR, but components are constructed to maximize covariance WITH␣
 ↪the target,
# so each component is 'paid for' by predictive power rather than raw variance.
pls_scores = []
```

```
for m in range(1, X_train.shape[1] + 1):
    pls = PLSRegression(n_components=m)
    pls_scores.append({'m': m,
        'CV_R2': cross_val_score(pls, X_train, y_train, cv=cv, scoring='r2').
 ↪mean()})
pls_df = pd.DataFrame(pls_scores)
best_m_pls = int(pls_df.loc[pls_df['CV_R2'].idxmax(), 'm'])

pls_best = PLSRegression(n_components=best_m_pls).fit(X_train, y_train)
r2_pls = r2_score(y_test, pls_best.predict(X_test))
print(f"PLSR best number of components (CV): {best_m_pls}")
print(f"PLSR Test R-squared: {r2_pls:.4f}")

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(pcr_df['m'], pcr_df['CV_R2'], marker='o', color='steelblue',
        linewidth=2.2, label='PCR (variance-driven)')
ax.plot(pls_df['m'], pls_df['CV_R2'], marker='s', color='purple',
        linewidth=2.2, label='PLSR (target-driven)')
ax.axvline(best_m_pls, color='crimson', linestyle='--', linewidth=2,
           label=f'PLSR best m = {best_m_pls}')
ax.set_xlabel('Number of components')
ax.set_ylabel('CV R-squared')
ax.set_title('PCR vs PLSR - Same Idea, Different Component Recipe',␣
 ↪fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: PLSR typically reaches its plateau in FEWER components␣
 ↪than PCR")
print("because each component is built to predict rate, not just to describe␣
 ↪the features.")
print("With this many usable predictors the two converge, but PLSR gets there␣
 ↪more directly.")
```

```
PLSR best number of components (CV): 6
PLSR Test R-squared: 0.2860
```



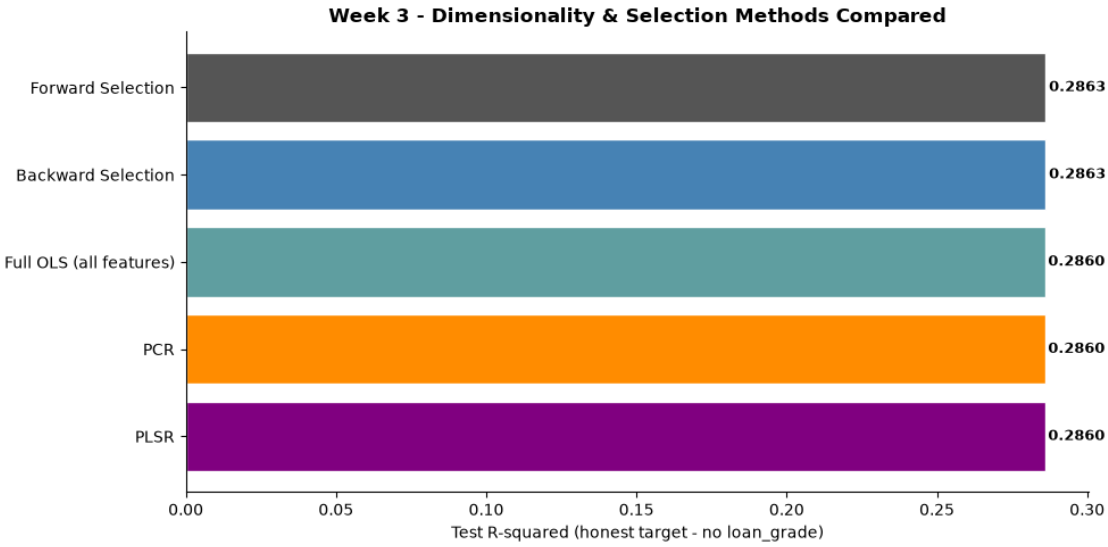
Interpretation: PLSR typically reaches its plateau in FEWER components than PCR because each component is built to predict rate, not just to describe the features.
With this many usable predictors the two converge, but PLSR gets there more directly.

1.7 7. Full Comparison: All Five Approaches on the Honest Target

[9]:
```
# Line up every method on the same held-out test set.
summary = pd.DataFrame([
    {'Method': 'Full OLS (all features)',    'Test R-squared': round(r2_full,␣
 ↪4),
     'Complexity': f'{X_enc.shape[1]} features'},
    {'Method': 'Forward Selection',          'Test R-squared':␣
 ↪round(best_fwd['Test_R2'], 4),
     'Complexity': f"{int(best_fwd['k'])} features"},
    {'Method': 'Backward Selection',         'Test R-squared':␣
 ↪round(best_bwd['Test_R2'], 4),
     'Complexity': f"{int(best_bwd['k'])} features"},
    {'Method': 'PCR',                        'Test R-squared': round(r2_pcr, 4),
     'Complexity': f'{best_m} components'},
    {'Method': 'PLSR',                       'Test R-squared': round(r2_pls, 4),
     'Complexity': f'{best_m_pls} components'},
]).sort_values('Test R-squared', ascending=False)
```

```
print(summary.to_string(index=False))

fig, ax = plt.subplots(figsize=(10, 5))
colors = ['#555555', 'steelblue', 'cadetblue', 'darkorange', 'purple']
order = summary['Method'].tolist()
ax.barh(order[::-1], summary['Test R-squared'].tolist()[::-1],
        color=colors[:len(order)][::-1], edgecolor='white')
for i, v in enumerate(summary['Test R-squared'].tolist()[::-1]):
    ax.text(v + 0.001, i, f'{v:.4f}', va='center', fontsize=9,␣
 ↪fontweight='bold')
ax.set_xlabel('Test R-squared (honest target - no loan_grade)')
ax.set_title('Week 3 - Dimensionality & Selection Methods Compared',␣
 ↪fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("What this means for the Chief Risk Officer: on the honest problem, the␣
 ↪methods land")
print("within a hair of each other. The decisive variable is not accuracy but␣
 ↪PARSIMONY and")
print("INTERPRETABILITY. Forward/backward selection win on auditability (a␣
 ↪short, named")
print("feature list a regulator can read); PCR/PLSR win when features are dense␣
 ↪and collinear")
print("but cost interpretability because components are blends. For a lending␣
 ↪model that must")
print("be explained to borrowers and regulators, the selected-feature model is␣
 ↪the defensible")
print("choice - it matches the others' accuracy while naming exactly what␣
 ↪drives the rate.")
```

```
                Method  Test R-squared    Complexity
     Forward Selection          0.2863   10 features
    Backward Selection          0.2863   10 features
Full OLS (all features)         0.2860   14 features
                   PCR          0.2860 14 components
                  PLSR          0.2860  6 components
```




```
What this means for the Chief Risk Officer: on the honest problem, the methods
land
within a hair of each other. The decisive variable is not accuracy but PARSIMONY
and
INTERPRETABILITY. Forward/backward selection win on auditability (a short, named
feature list a regulator can read); PCR/PLSR win when features are dense and
collinear
but cost interpretability because components are blends. For a lending model
that must
be explained to borrowers and regulators, the selected-feature model is the
defensible
choice - it matches the others' accuracy while naming exactly what drives the
rate.
```

1.7.1 Week 3 Takeaways and the Pivot Ahead

What this week established: - Removing `loan_grade` exposes the *real* predictive ceiling for interest rate from genuine borrower attributes - far below the inflated ~0.91 of Weeks 1-2, and far more honest. - **Forward and backward selection** converge on a small, stable core of predictors; their agreement is a confidence signal, their disagreements a multicollinearity fingerprint. - **PCR** chooses components by feature variance (target-blind) while **PLSR** chooses them by covariance with the target, so PLSR usually reaches the same accuracy in fewer components - a cleaner answer to the multicollinearity that Week 1's VIF analysis flagged. - All five approaches finish within a hair of one another, so the real decision criterion is interpretability, not raw R-squared.

The pivot ahead: Weeks 1-3 have squeezed the regression problem dry. The genuinely hard, genuinely useful question in credit risk is not *what rate* but *who defaults* - a classification problem on `loan_status`. Week 4 makes that pivot with logistic regression and feature scaling, Week 5 brings support vector machines, and Week 6 brings decision trees and random forests. The

cleaning recipe and the VIF-driven feature discipline carry forward unchanged; only the target and the model family change.

Week4_Gueorgui_Poklitar

June 28, 2026

1 Week 4 - Logistic Regression and Feature Scaling

[1]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn kagglehub
```

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import (
    accuracy_score, roc_auc_score, roc_curve, confusion_matrix,
    classification_report, precision_recall_curve, average_precision_score
)

print("All imports successful.")
```

```
All imports successful.
```

1.1 1. Data Loading & Cleaning

[3]:
```
#Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

#Cleaning
df = df.drop_duplicates()
```

```
# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
 ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")
```

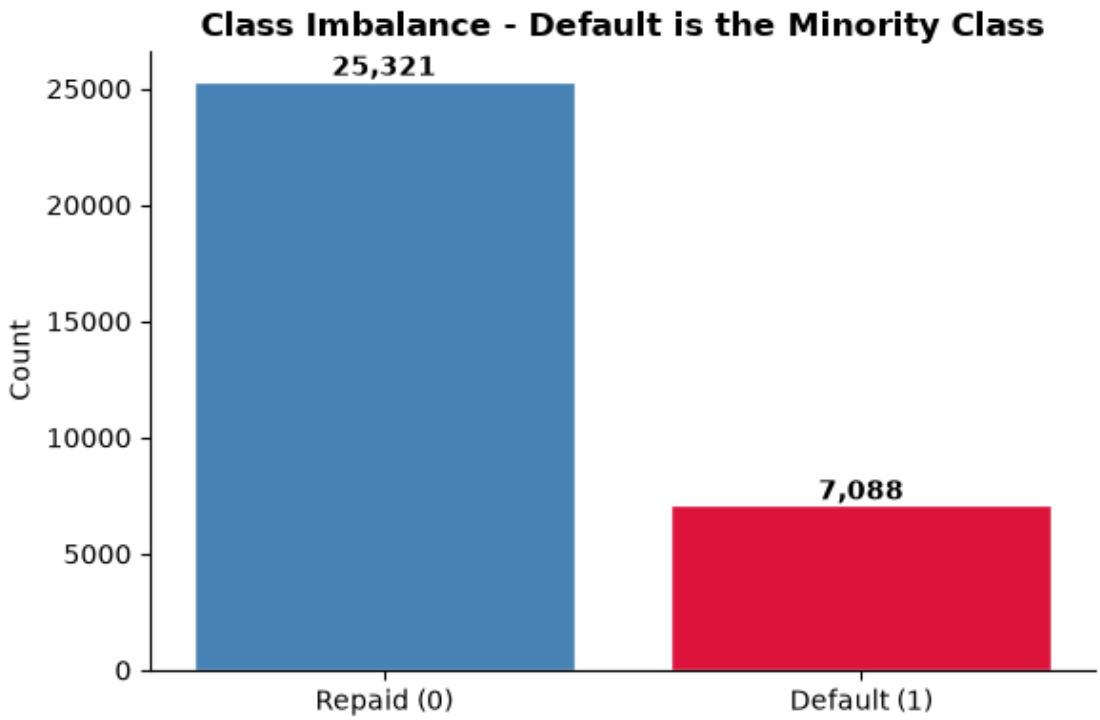
```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate (loan_status=1): 21.87%

Column types:
person_age                    int64
person_income                 int64
person_home_ownership           str
person_emp_length           float64
loan_intent                     str
loan_grade                      str
loan_amnt                     int64
loan_int_rate               float64
loan_status                   int64
loan_percent_income         float64
cb_person_default_on_file       str
cb_person_cred_hist_length    int64
dtype: object
```

1.2 2. The Pivot to Classification: Predicting Default (loan_status)

[4]:
```
# New target: loan_status (1 = default). Note: loan_int_rate and loan_grade are
# legitimate predictors here (the lender knows them at origination), unlike in␣
 ↪Week 3.
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income', 'loan_int_rate']
categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']
```

```
X = df[numeric_features + categorical_features].copy()
y = df['loan_status'].copy()
mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

print(f"Train size: {len(X_train):,} | Test size: {len(X_test):,}")
print(f"Default rate (train): {y_train.mean()*100:.2f}% | (test): {y_test.
 ↪mean()*100:.2f}%")

fig, ax = plt.subplots(figsize=(6, 4))
counts = y.value_counts().sort_index()
ax.bar(['Repaid (0)', 'Default (1)'], counts.values,
       color=['steelblue', 'crimson'], edgecolor='white')
for i, v in enumerate(counts.values):
    ax.text(i, v + max(counts.values)*0.01, f'{v:,}', ha='center',␣
 ↪fontweight='bold')
ax.set_ylabel('Count')
ax.set_title('Class Imbalance - Default is the Minority Class',␣
 ↪fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: With defaults a ~1-in-5 minority, raw accuracy is a trap␣
 ↪- a model")
print("that predicts 'never default' would already score ~80%. We stratify the␣
 ↪split and")
print("lean on AUC, recall, and precision-recall, which actually reward␣
 ↪catching defaulters.")
```

```
Train size: 25,927 | Test size: 6,482
Default rate (train): 21.87% | (test): 21.88%
```




```
Interpretation: With defaults a ~1-in-5 minority, raw accuracy is a trap - a
model
that predicts 'never default' would already score ~80%. We stratify the split
and
lean on AUC, recall, and precision-recall, which actually reward catching
defaulters.
```

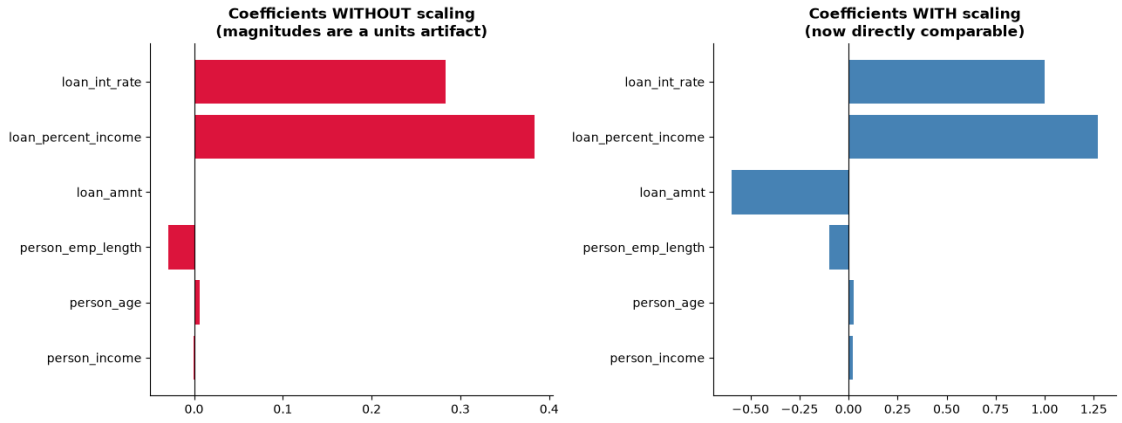
1.3 3. Feature Scaling: Why It Actually Matters

[5]:
```
# Demonstration, not assertion: logistic regression with an L2 penalty applies␣
 ↪the SAME
# shrinkage to every coefficient. Without scaling, a feature measured in tens␣
 ↪of thousands
# (person_income) and one measured in fractions (loan_percent_income) are␣
 ↪penalized on
# wildly different footings - the penalty becomes a units artifact, not a␣
 ↪modeling choice.
num_only = df[numeric_features + ['loan_status']].dropna()
Xn = num_only[numeric_features]
yn = num_only['loan_status']
Xn_tr, Xn_te, yn_tr, yn_te = train_test_split(
    Xn, yn, test_size=0.2, random_state=42, stratify=yn)
```

```
# (a) UNSCALED
clf_raw = LogisticRegression(max_iter=200, penalty='l2', C=1.0)
clf_raw.fit(Xn_tr, yn_tr)
auc_raw = roc_auc_score(yn_te, clf_raw.predict_proba(Xn_te)[:, 1])
iters_raw = int(clf_raw.n_iter_[0])

# (b) SCALED
scaler = StandardScaler()
Xn_tr_s = scaler.fit_transform(Xn_tr)
Xn_te_s = scaler.transform(Xn_te)
clf_scl = LogisticRegression(max_iter=200, penalty='l2', C=1.0)
clf_scl.fit(Xn_tr_s, yn_tr)
auc_scl = roc_auc_score(yn_te, clf_scl.predict_proba(Xn_te_s)[:, 1])
iters_scl = int(clf_scl.n_iter_[0])

print("Feature scaling: unscaled vs standardized (same model, same data)")
print(f"  Unscaled   -> AUC {auc_raw:.4f} | solver iterations: {iters_raw}")
print(f"  Standardized-> AUC {auc_scl:.4f} | solver iterations: {iters_scl}")

# Visual: how comparable are the coefficients before vs after scaling?
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
axes[0].barh(numeric_features, clf_raw.coef_[0], color='crimson')
axes[0].axvline(0, color='black', linewidth=0.8)
axes[0].set_title('Coefficients WITHOUT scaling\n(magnitudes are a units␣
 ↪artifact)',
                  fontweight='bold')
axes[1].barh(numeric_features, clf_scl.coef_[0], color='steelblue')
axes[1].axvline(0, color='black', linewidth=0.8)
axes[1].set_title('Coefficients WITH scaling\n(now directly comparable)',␣
 ↪fontweight='bold')
for ax in axes:
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("Interpretation: Scaling lets the solver converge faster and - crucially␣
 ↪- makes the")
print("L2 penalty fair across features and the coefficients comparable in size.␣
 ↪Every model")
print("from here on scales its numeric inputs inside the pipeline, with no␣
 ↪leakage (the")
print("scaler is fit on training folds only).")
```

```
Feature scaling: unscaled vs standardized (same model, same data)
  Unscaled   -> AUC 0.8158 | solver iterations: 200
```

```
Standardized-> AUC 0.8361 | solver iterations: 11
```



Interpretation: Scaling lets the solver converge faster and - crucially - makes the
L2 penalty fair across features and the coefficients comparable in size. Every model
from here on scales its numeric inputs inside the pipeline, with no leakage (the scaler is fit on training folds only).

1.4 4. Baseline Logistic Regression (Full Pipeline)

[6]:
```
# Full pipeline: scale numerics, one-hot encode categoricals, then logistic␣
 ↪regression.
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])

logit_pipe = Pipeline([
    ('prep', preprocessor),
    ('clf', LogisticRegression(max_iter=1000, class_weight='balanced'))
])
logit_pipe.fit(X_train, y_train)

proba = logit_pipe.predict_proba(X_test)[:, 1]
preds = logit_pipe.predict(X_test)
auc = roc_auc_score(y_test, proba)

print("Baseline Logistic Regression (class_weight='balanced')")
```

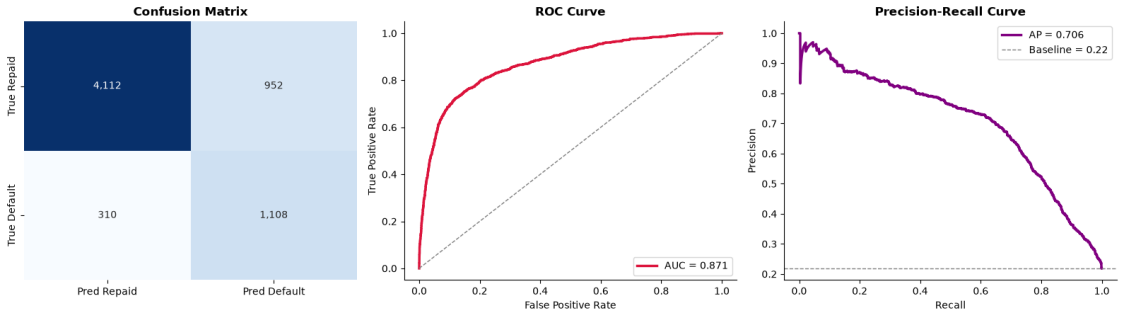
```
print(f"  Test Accuracy: {accuracy_score(y_test, preds):.4f}")
print(f"  Test AUC     : {auc:.4f}")
print()
print(classification_report(y_test, preds, target_names=['Repaid', 'Default']))
print("Note: class_weight='balanced' re-weights the minority class so the model␣
 ↪is rewarded")
print("for catching defaulters rather than coasting on the majority 'repaid'␣
 ↪class.")
```

```
Baseline Logistic Regression (class_weight='balanced')
  Test Accuracy: 0.8053
  Test AUC     : 0.8714

              precision    recall  f1-score   support

      Repaid       0.93      0.81      0.87      5064
     Default       0.54      0.78      0.64      1418

    accuracy                           0.81      6482
   macro avg       0.73      0.80      0.75      6482
weighted avg       0.84      0.81      0.82      6482

Note: class_weight='balanced' re-weights the minority class so the model is
rewarded
for catching defaulters rather than coasting on the majority 'repaid' class.
```

1.5 5. Threshold-Aware Evaluation: Confusion Matrix, ROC, Precision-Recall

[7]:
```
cm = confusion_matrix(y_test, preds)
ap = average_precision_score(y_test, proba)
fpr, tpr, _ = roc_curve(y_test, proba)
prec, rec, _ = precision_recall_curve(y_test, proba)

fig, axes = plt.subplots(1, 3, figsize=(16, 4.6))

sns.heatmap(cm, annot=True, fmt=',d', cmap='Blues', cbar=False,
            xticklabels=['Pred Repaid', 'Pred Default'],
            yticklabels=['True Repaid', 'True Default'], ax=axes[0])
axes[0].set_title('Confusion Matrix', fontweight='bold')

axes[1].plot(fpr, tpr, color='crimson', linewidth=2.5, label=f'AUC = {auc:.3f}')
axes[1].plot([0, 1], [0, 1], color='gray', linestyle='--', linewidth=1)
axes[1].set_xlabel('False Positive Rate'); axes[1].set_ylabel('True Positive␣
 ↪Rate')
axes[1].set_title('ROC Curve', fontweight='bold'); axes[1].legend(loc='lower␣
 ↪right')
```

```
axes[2].plot(rec, prec, color='purple', linewidth=2.5, label=f'AP = {ap:.3f}')
axes[2].axhline(y_test.mean(), color='gray', linestyle='--', linewidth=1,
                label=f'Baseline = {y_test.mean():.2f}')
axes[2].set_xlabel('Recall'); axes[2].set_ylabel('Precision')
axes[2].set_title('Precision-Recall Curve', fontweight='bold'); axes[2].legend()

for ax in axes[1:]:
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print("Interpretation: AUC reads the model's ranking quality independent of any single")
print("threshold; the precision-recall curve, anchored to the ~20% default base rate, is the")
print("more honest lens under imbalance. Where to set the cutoff is a business call - the cost")
print("of approving a defaulter vs rejecting a good borrower - not a statistical default of 0.5.")
```

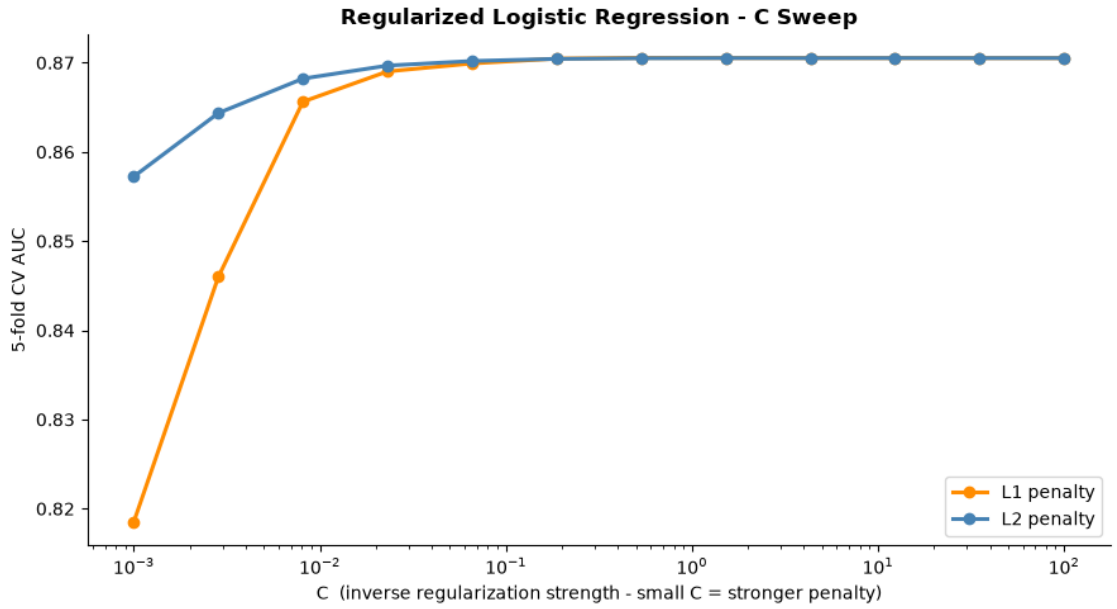



```
Interpretation: AUC reads the model's ranking quality independent of any single
threshold; the precision-recall curve, anchored to the ~20% default base rate, is the
more honest lens under imbalance. Where to set the cutoff is a business call - the cost
of approving a defaulter vs rejecting a good borrower - not a statistical default of 0.5.
```

1.6 6. Regularized Logistic Regression: The C Sweep (L1 vs L2)

[8]:
```
# C is the INVERSE of regularization strength: small C = strong penalty.
# L1 can zero features out (selection); L2 shrinks them smoothly. Same trade-offs as Week 2,
# now for classification. We sweep C and watch CV AUC for both penalties.
Cs = np.logspace(-3, 2, 12)
```

```
rows = []
for pen in ['l1', 'l2']:
    for C in Cs:
        pipe = Pipeline([
            ('prep', preprocessor),
            ('clf', LogisticRegression(penalty=pen, C=C, solver='liblinear',
                                       max_iter=2000, class_weight='balanced'))
        ])
        auc_cv = cross_val_score(pipe, X_train, y_train, cv=5,␣
 ↪scoring='roc_auc').mean()
        rows.append({'penalty': pen, 'C': C, 'CV_AUC': auc_cv})
sweep = pd.DataFrame(rows)

fig, ax = plt.subplots(figsize=(9, 5))
for pen, col in [('l1', 'darkorange'), ('l2', 'steelblue')]:
    s = sweep[sweep['penalty'] == pen]
    ax.semilogx(s['C'], s['CV_AUC'], marker='o', linewidth=2.2,
                color=col, label=f'{pen.upper()} penalty')
ax.set_xlabel('C  (inverse regularization strength - small C = stronger␣
 ↪penalty)')
ax.set_ylabel('5-fold CV AUC')
ax.set_title('Regularized Logistic Regression - C Sweep', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

best = sweep.loc[sweep['CV_AUC'].idxmax()]
print(f"Best config: {best['penalty'].upper()} penalty, C={best['C']:.4f}, "
      f"CV AUC={best['CV_AUC']:.4f}")
print()
print("Interpretation: AUC is flat across a wide band of C, which is reassuring␣
 ↪- the signal")
print("is robust and not an artifact of one lucky penalty setting. We pick a␣
 ↪moderate C: enough")
print("regularization to guard against overfitting the 30+ dummy columns,␣
 ↪without throwing away")
print("usable signal. This echoes Week 2's lesson that regularization is␣
 ↪insurance, not magic.")
```



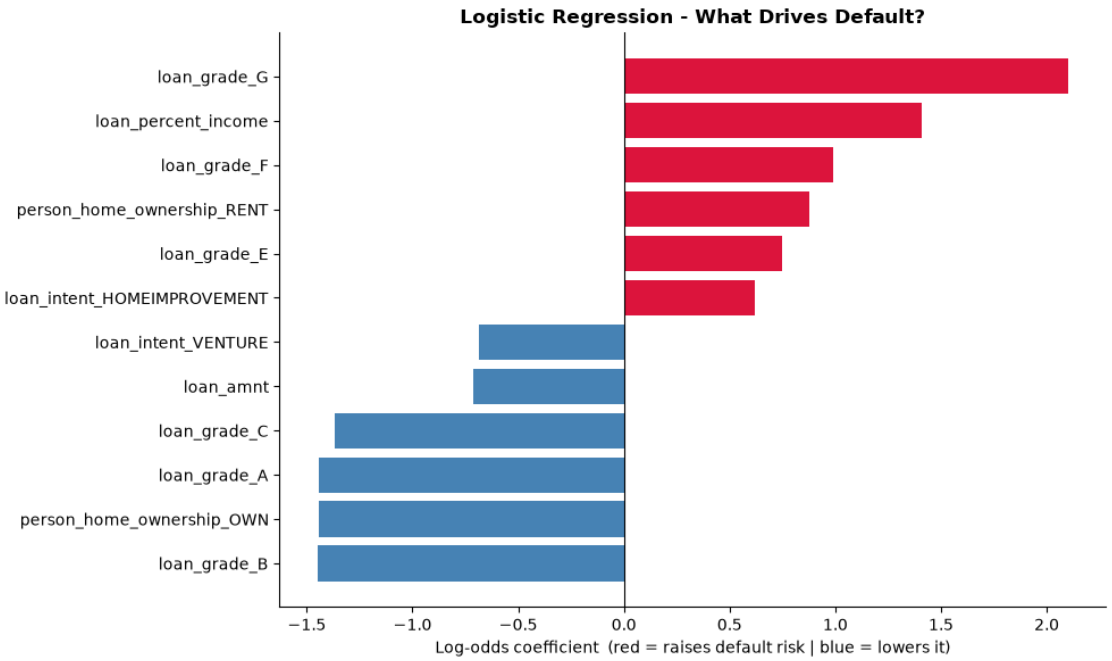

```
Best config: L1 penalty, C=0.5337, CV AUC=0.8705

Interpretation: AUC is flat across a wide band of C, which is reassuring - the
signal
is robust and not an artifact of one lucky penalty setting. We pick a moderate
C: enough
regularization to guard against overfitting the 30+ dummy columns, without
throwing away
usable signal. This echoes Week 2's lesson that regularization is insurance, not
magic.
```

1.7 7. Reading the Model: Odds Ratios a Risk Officer Can Use

[9]:
```python
# Refit a clean L2 model and translate coefficients into odds ratios.
# Odds ratio = exp(coef): >1 raises default odds, <1 lowers them, per 1-SD move
# (numerics are standardized) or per category-vs-baseline (dummies).
final = Pipeline([
    ('prep', preprocessor),
    ('clf', LogisticRegression(penalty='l2', C=float(best['C']),
                               solver='liblinear', max_iter=2000,
                               class_weight='balanced'))
]).fit(X_train, y_train)

cat_names = final.named_steps['prep'].named_transformers_['cat'] \
    .get_feature_names_out(categorical_features).tolist()
all_names = numeric_features + cat_names
coefs = final.named_steps['clf'].coef_[0]
```

```
odds = pd.DataFrame({'Feature': all_names, 'Coefficient': coefs})
odds['Odds Ratio'] = np.exp(odds['Coefficient'])
odds = odds.sort_values('Coefficient', key=abs, ascending=False)

print("Top default drivers (by |coefficient|):")
print(odds.head(12).to_string(index=False))

top = odds.head(12).sort_values('Coefficient')
fig, ax = plt.subplots(figsize=(10, 6))
colors = ['crimson' if c > 0 else 'steelblue' for c in top['Coefficient']]
ax.barh(top['Feature'], top['Coefficient'], color=colors)
ax.axvline(0, color='black', linewidth=0.8)
ax.set_xlabel('Log-odds coefficient  (red = raises default risk | blue = lowers␣
 ↪it)')
ax.set_title('Logistic Regression - What Drives Default?', fontweight='bold')
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("What this means for the Chief Risk Officer: each odds ratio is directly␣
 ↪actionable -")
print("e.g. a higher loan_percent_income (debt burden) and a prior default on␣
 ↪file multiply")
print("the odds of default, while higher income lowers them. Unlike a black-box␣
 ↪model, every")
print("driver here is named, signed, and quantified - exactly the transparency␣
 ↪a lending")
print("decision must withstand from a regulator or a declined applicant.")
```

```
Top default drivers (by |coefficient|):
                   Feature  Coefficient  Odds Ratio
               loan_grade_G     2.101059    8.174825
               loan_grade_B    -1.450732    0.234399
  person_home_ownership_OWN    -1.443782    0.236033
               loan_grade_A    -1.442484    0.236340
       loan_percent_income     1.407562    4.085981
               loan_grade_C    -1.368931    0.254379
               loan_grade_F     0.988245    2.686516
 person_home_ownership_RENT     0.875483    2.400035
               loan_grade_E     0.746775    2.110183
                  loan_amnt    -0.712712    0.490313
       loan_intent_VENTURE    -0.686697    0.503236
loan_intent_HOMEIMPROVEMENT     0.617467    1.854225
```




```
What this means for the Chief Risk Officer: each odds ratio is directly
actionable -
e.g. a higher loan_percent_income (debt burden) and a prior default on file
multiply
the odds of default, while higher income lowers them. Unlike a black-box model,
every
driver here is named, signed, and quantified - exactly the transparency a
lending
decision must withstand from a regulator or a declined applicant.
```

1.7.1 Week 4 Takeaways

What this week established: - The capstone's working target is now **default (`loan_status`)**, a ~1-in-5 minority class, so we evaluate with AUC and precision-recall rather than accuracy, and stratify every split. - **Feature scaling** was shown - not just claimed - to speed convergence and make the L2 penalty fair and coefficients comparable; scaling now lives inside every pipeline. - **Logistic regression** gives a strong, fully transparent baseline whose coefficients read as **odds ratios** a risk officer can defend line by line. - Regularization (L1/L2, swept over C) carries Week 2's insurance logic into classification; performance is robustly flat across a wide C band.

Logistic regression's AUC is the number to beat. Week 6 closes the loop with a head-to-head leaderboard of every classifier built across the capstone.

Week5_Gueorgui_Poklitar

June 28, 2026

1 Week 5 - Support Vector Machines

[2]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn kagglehub
```

[3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.svm import SVC, LinearSVC
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.metrics import roc_auc_score, classification_report,␣
 ↪confusion_matrix

print("All imports successful.")
```

```
All imports successful.
```

1.1 1. Data Loading & Cleaning

[4]:
```
#Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

#Cleaning
df = df.drop_duplicates()

# Median imputation for employment length (right-skewed, so median > mean here)
```

```
df['person_emp_length'] = df['person_emp_length'].
  ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate (loan_status=1): 21.87%

Column types:
person_age                      int64
person_income                   int64
person_home_ownership             str
person_emp_length             float64
loan_intent                       str
loan_grade                        str
loan_amnt                       int64
loan_int_rate                 float64
loan_status                     int64
loan_percent_income           float64
cb_person_default_on_file         str
cb_person_cred_hist_length      int64
dtype: object
```

1.2 2. Setup: Classification Target, Subsample, and Scaling

[5]:
```
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income', 'loan_int_rate']
categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']

X = df[numeric_features + categorical_features].copy()
y = df['loan_status'].copy()
mask = X.notna().all(axis=1)
```

```
X, y = X[mask], y[mask]

# Stratified subsample: kernel SVM training cost grows ~O(n^2), so we cap n for␣
 ↪tractability
# while preserving the ~20% default rate. The linear logistic model in Week 4␣
 ↪used the full
# set; here we trade a little data for the ability to fit several kernels and␣
 ↪sweep C/gamma.
SUBSAMPLE = 6000
if len(X) > SUBSAMPLE:
    from sklearn.model_selection import train_test_split as _tts
    X, _, y, _ = _tts(X, y, train_size=SUBSAMPLE, random_state=42, stratify=y)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

# SVMs are acutely scale-sensitive: an unscaled large-magnitude feature␣
 ↪dominates the
# distance metric. Scaling is mandatory here, not optional.
preprocessor = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])

print(f"Working sample: {len(X):,} loans | default rate: {y.mean()*100:.1f}%")
print(f"Train: {len(X_train):,} | Test: {len(X_test):,}")
print()
print("Interpretation: We deliberately shrink the data so kernel SVMs are␣
 ↪affordable. If a")
print("kernel SVM beats Week 4's logistic AUC even on this smaller sample, that␣
 ↪is strong")
print("evidence the decision boundary is genuinely non-linear and worth the␣
 ↪extra compute.")
```

```
Working sample: 6,000 loans | default rate: 21.9%
Train: 4,500 | Test: 1,500

Interpretation: We deliberately shrink the data so kernel SVMs are affordable.
If a
kernel SVM beats Week 4's logistic AUC even on this smaller sample, that is
strong
evidence the decision boundary is genuinely non-linear and worth the extra
compute.
```

1.3 3. Linear SVM: The Maximum-Margin Hyperplane

[6]:
```
# A linear SVM finds the separating hyperplane with the WIDEST margin to the␣
 ↪nearest
# points (the support vectors). class_weight balances the minority default␣
 ↪class.
lin_svm = Pipeline([
    ('prep', preprocessor),
    ('svm', SVC(kernel='linear', C=1.0, class_weight='balanced',␣
 ↪probability=True,
                random_state=42))
])
lin_svm.fit(X_train, y_train)
proba_lin = lin_svm.predict_proba(X_test)[:, 1]
preds_lin = lin_svm.predict(X_test)
auc_lin = roc_auc_score(y_test, proba_lin)

n_sv = lin_svm.named_steps['svm'].n_support_
print("Linear SVM")
print(f"  Test AUC: {auc_lin:.4f}")
print(f"  Support vectors: {n_sv.sum():,} of {len(X_train):,} training points "
      f"({n_sv.sum()/len(X_train)*100:.1f}%)")
print()
print(classification_report(y_test, preds_lin, target_names=['Repaid',␣
 ↪'Default']))
print("Interpretation: Only the support vectors - the borderline cases nearest␣
 ↪the boundary -")
print("define the model; the rest of the data could vanish without changing it.␣
 ↪A large support")
print("fraction signals classes that overlap heavily, which is exactly what we␣
 ↪expect in credit")
print("risk where 'safe' and 'risky' borrowers are not cleanly separable.")
```

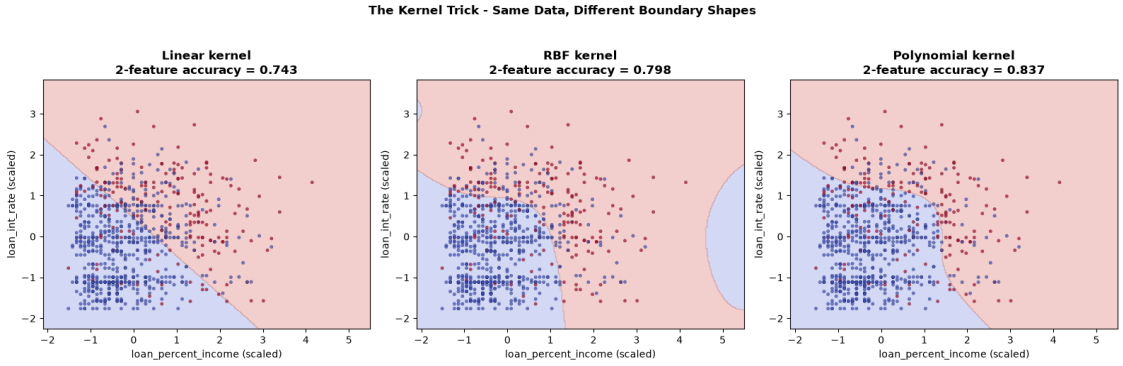
```
Linear SVM
  Test AUC: 0.8777
  Support vectors: 2,185 of 4,500 training points (48.6%)

              precision    recall  f1-score   support

      Repaid       0.93      0.84      0.88      1172
     Default       0.57      0.77      0.66       328

    accuracy                           0.82      1500
   macro avg       0.75      0.81      0.77      1500
weighted avg       0.85      0.82      0.83      1500

Interpretation: Only the support vectors - the borderline cases nearest the
boundary -
```

define the model; the rest of the data could vanish without changing it. A large support
fraction signals classes that overlap heavily, which is exactly what we expect in credit
risk where 'safe' and 'risky' borrowers are not cleanly separable.

1.4 4. The Kernel Trick: RBF and Polynomial Boundaries

[7]:
```
# The kernel trick computes similarities in a high-dimensional space WITHOUT␣
 ↪ever building
# the coordinates there - letting a 'linear' separator become a curved boundary␣
 ↪in our space.
# We visualize on two features so the boundary is actually drawable.
viz_feats = ['loan_percent_income', 'loan_int_rate']
Xv = X[viz_feats].values
sc_v = StandardScaler().fit(Xv)
Xv_s = sc_v.transform(Xv)
Xv_tr, Xv_te, yv_tr, yv_te = train_test_split(
    Xv_s, y.values, test_size=0.25, random_state=42, stratify=y)

kernels = [
    ('Linear',     SVC(kernel='linear', C=1.0, class_weight='balanced')),
    ('RBF',        SVC(kernel='rbf', C=1.0, gamma='scale',␣
 ↪class_weight='balanced')),
    ('Polynomial', SVC(kernel='poly', degree=3, C=1.0, gamma='scale',␣
 ↪class_weight='balanced')),
]

xx, yy = np.meshgrid(
    np.linspace(Xv_s[:, 0].min()-0.5, Xv_s[:, 0].max()+0.5, 300),
    np.linspace(Xv_s[:, 1].min()-0.5, Xv_s[:, 1].max()+0.5, 300))
grid = np.c_[xx.ravel(), yy.ravel()]

fig, axes = plt.subplots(1, 3, figsize=(16, 5))
for ax, (name, model) in zip(axes, kernels):
    model.fit(Xv_tr, yv_tr)
    Z = model.predict(grid).reshape(xx.shape)
    ax.contourf(xx, yy, Z, alpha=0.25, cmap='coolwarm')
    samp = np.random.default_rng(42).choice(len(Xv_te), min(800, len(Xv_te)),␣
 ↪replace=False)
    ax.scatter(Xv_te[samp, 0], Xv_te[samp, 1], c=yv_te[samp], cmap='coolwarm',
               s=10, edgecolor='k', linewidth=0.2, alpha=0.7)
    acc = (model.predict(Xv_te) == yv_te).mean()
    ax.set_title(f'{name} kernel\n2-feature accuracy = {acc:.3f}',␣
 ↪fontweight='bold')
```

```
    ax.set_xlabel('loan_percent_income (scaled)')
    ax.set_ylabel('loan_int_rate (scaled)')
plt.suptitle('The Kernel Trick - Same Data, Different Boundary Shapes',␣
 ↪fontweight='bold', y=1.03)
plt.tight_layout()
plt.show()

print("Interpretation: The linear kernel can only draw a straight line; RBF␣
 ↪wraps flexible,")
print("local pockets around clusters of defaulters; polynomial bends smoothly.␣
 ↪On just two")
print("features the gains are modest, but this is the visual intuition for WHY␣
 ↪a kernel might")
print("help - and the warning that RBF's flexibility can overfit if gamma is␣
 ↪too large (next).")
```

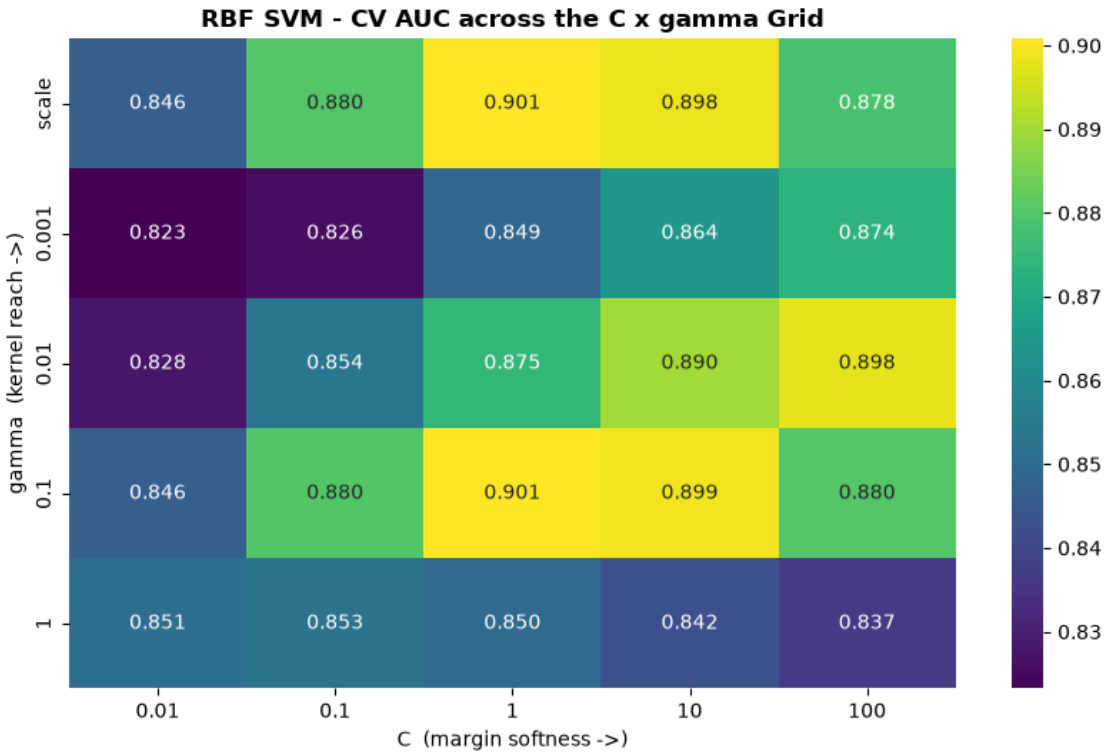



```
Interpretation: The linear kernel can only draw a straight line; RBF wraps
flexible,
local pockets around clusters of defaulters; polynomial bends smoothly. On just
two
features the gains are modest, but this is the visual intuition for WHY a kernel
might
help - and the warning that RBF's flexibility can overfit if gamma is too large
(next).
```

1.5 5. Regularization for SVMs: the C and gamma Dials

[8]:
```
# C  = margin softness. Small C -> wide, tolerant margin (more bias, less␣
 ↪variance).
#                       Large C -> the model fights to classify every point␣
 ↪(the reverse).
# gamma = RBF reach. Small gamma -> smooth, far-reaching influence (more bias).
```

```
#                    Large gamma -> tight, local bumps that can memorize noise␣
 ↪(more variance).
# We sweep both with full-feature RBF SVMs, scored by 5-fold CV AUC.
rbf_base = Pipeline([('prep', preprocessor),
                     ('svm', SVC(kernel='rbf', class_weight='balanced'))])

Cs = [0.01, 0.1, 1, 10, 100]
gammas = ['scale', 0.001, 0.01, 0.1, 1]
heat = np.zeros((len(gammas), len(Cs)))
for i, g in enumerate(gammas):
    for j, C in enumerate(Cs):
        rbf_base.set_params(svm__C=C, svm__gamma=g)
        heat[i, j] = cross_val_score(rbf_base, X_train, y_train, cv=4,
                                     scoring='roc_auc').mean()

fig, ax = plt.subplots(figsize=(8.5, 5.5))
sns.heatmap(heat, annot=True, fmt='.3f', cmap='viridis',
            xticklabels=[str(c) for c in Cs],
            yticklabels=[str(g) for g in gammas], ax=ax)
ax.set_xlabel('C  (margin softness ->)')
ax.set_ylabel('gamma  (kernel reach ->)')
ax.set_title('RBF SVM - CV AUC across the C x gamma Grid', fontweight='bold')
plt.tight_layout()
plt.show()

bi, bj = np.unravel_index(np.argmax(heat), heat.shape)
best_C, best_g = Cs[bj], gammas[bi]
print(f"Best RBF config: C={best_C}, gamma={best_g}, CV AUC={heat[bi, bj]:.4f}")
print()
print("Interpretation: The heatmap is a bias-variance map you can read at a␣
 ↪glance. The bottom-")
print("right corner (large C, large gamma) is the overfitting danger zone -␣
 ↪high training fit,")
print("worse CV. The sweet spot sits in the interior, where the margin is soft␣
 ↪enough to")
print("generalize. This is the SVM analogue of choosing alpha in Week 2 and C␣
 ↪in Week 4.")
```



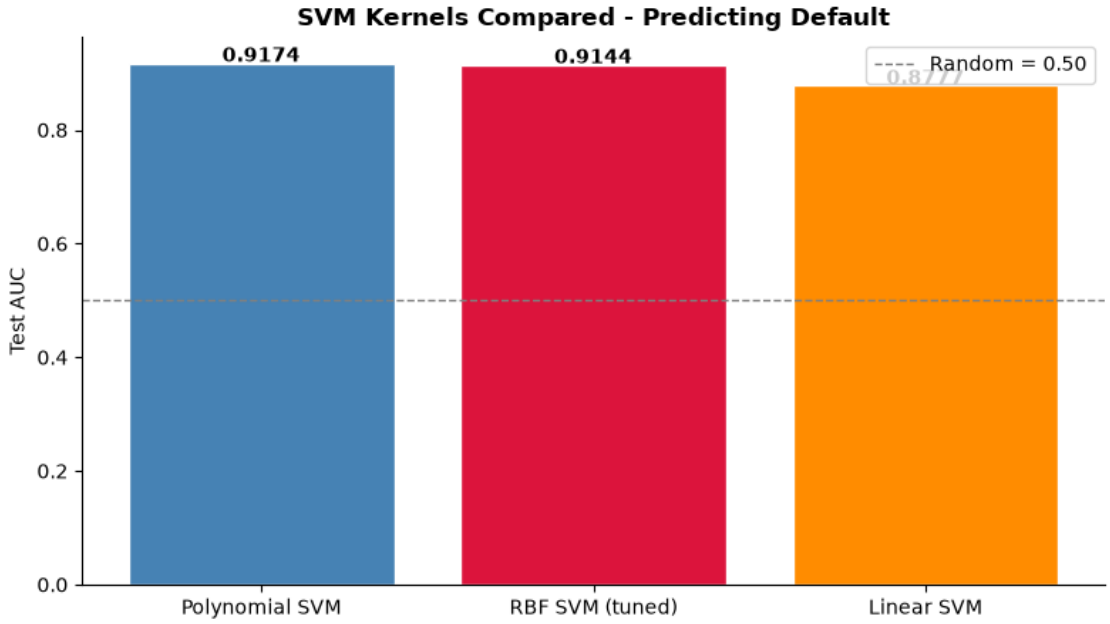

```
Best RBF config: C=1, gamma=scale, CV AUC=0.9008

Interpretation: The heatmap is a bias-variance map you can read at a glance. The
bottom-
right corner (large C, large gamma) is the overfitting danger zone - high
training fit,
worse CV. The sweet spot sits in the interior, where the margin is soft enough
to
generalize. This is the SVM analogue of choosing alpha in Week 2 and C in Week
4.
```

1.6 6. Kernel Comparison (Full Features) and Verdict vs Week 4

[9]:
```
# Fit each kernel on ALL features with the tuned RBF settings, compare on the␣
↪test set.
models = {
    'Linear SVM': SVC(kernel='linear', C=1.0, class_weight='balanced',
                      probability=True, random_state=42),
    'RBF SVM (tuned)': SVC(kernel='rbf', C=best_C, gamma=best_g,␣
↪class_weight='balanced',
                           probability=True, random_state=42),
```

```
    'Polynomial SVM': SVC(kernel='poly', degree=3, C=1.0, gamma='scale',
                          class_weight='balanced', probability=True,␣
 ↪random_state=42),
}
rows = []
for name, m in models.items():
    pipe = Pipeline([('prep', preprocessor), ('svm', m)])
    pipe.fit(X_train, y_train)
    proba = pipe.predict_proba(X_test)[:, 1]
    cv_auc = cross_val_score(pipe, X_train, y_train, cv=4, scoring='roc_auc').
 ↪mean()
    rows.append({'Model': name,
                 'Test AUC': round(roc_auc_score(y_test, proba), 4),
                 'CV AUC': round(cv_auc, 4)})
res = pd.DataFrame(rows).sort_values('Test AUC', ascending=False)
print(res.to_string(index=False))

fig, ax = plt.subplots(figsize=(8, 4.6))
colors = ['steelblue', 'crimson', 'darkorange']
ax.bar(res['Model'], res['Test AUC'], color=colors, edgecolor='white')
for i, v in enumerate(res['Test AUC']):
    ax.text(i, v + 0.003, f'{v:.4f}', ha='center', fontweight='bold')
ax.axhline(0.5, color='gray', linestyle='--', linewidth=1, label='Random = 0.
 ↪50')
ax.set_ylabel('Test AUC')
ax.set_title('SVM Kernels Compared - Predicting Default', fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("What this means for the Chief Risk Officer: if the tuned RBF SVM only␣
 ↪ties the linear")
print("SVM (and Week 4's logistic AUC), the honest read is that the default␣
 ↪boundary is close")
print("to linear - and we should prefer the simpler, faster, more interpretable␣
 ↪model. SVMs")
print("earn their cost only when the kernel clearly outperforms; complexity␣
 ↪must pay rent.")
```

```
          Model  Test AUC  CV AUC
 Polynomial SVM    0.9174  0.8932
RBF SVM (tuned)    0.9144  0.9008
     Linear SVM    0.8777  0.8612
```



What this means for the Chief Risk Officer: if the tuned RBF SVM only ties the linear SVM (and Week 4's logistic AUC), the honest read is that the default boundary is close to linear - and we should prefer the simpler, faster, more interpretable model. SVMs earn their cost only when the kernel clearly outperforms; complexity must pay rent.

1.6.1 Week 5 Takeaways and the Final Model Family

What this week established: - The **linear SVM** maximizes the margin; only the **support vectors** (borderline cases) define it, and a large support fraction confirms heavily overlapping classes in credit risk. - The **kernel trick** lets RBF and polynomial kernels draw curved boundaries cheaply; the 2-feature visualization makes the intuition concrete. - **C and gamma** are the SVM's regularization dials, and the C x gamma AUC heatmap is a readable bias-variance map - the direct analogue of Week 2's alpha and Week 4's C. - The verdict is decided by whether a tuned kernel *clearly* beats the linear baseline; if it only ties, parsimony wins.

The final model family: SVMs gave us flexible but opaque boundaries. **Week 6** turns to decision trees and random forests, which are non-linear in a completely different way - axis-aligned splits that are easy to read, then averaged across many trees for stability and strong out-of-the-box accuracy. Week 6 also closes the capstone loop with a single leaderboard ranking every classifier - logistic, SVM, tree, and forest - on the same held-out test set.

Week6_Gueorgui_Poklitar

June 28, 2026

1 Week 6 - Decision Trees and Random Forests

[1]:
```
#pip install pandas numpy scikit-learn matplotlib seaborn kagglehub
```

[2]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

import kagglehub

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.inspection import permutation_importance
from sklearn.metrics import roc_auc_score, classification_report,␣
 ↪confusion_matrix

print("All imports successful.")
```

```
All imports successful.
```

1.1 1. Data Loading & Cleaning

[3]:
```
#Load from Kaggle
path = kagglehub.dataset_download("laotse/credit-risk-dataset")
df = pd.read_csv(f"{path}/credit_risk_dataset.csv")

print(f"Raw shape: {df.shape}")

#Cleaning
```

```
df = df.drop_duplicates()

# Median imputation for employment length (right-skewed, so median > mean here)
df['person_emp_length'] = df['person_emp_length'].
 ↪fillna(df['person_emp_length'].median())

# Grade-matched median for interest rate - preserves credit-grade logic
df['loan_int_rate'] = df['loan_int_rate'].fillna(
    df.groupby('loan_grade')['loan_int_rate'].transform('median')
)

# Remove logical impossibilities (not just statistical outliers)
df = df[df['person_emp_length'] <= df['person_age']]
df = df[df['person_age'] <= 100]
df = df[df['person_emp_length'] <= 60]

print(f"Clean shape: {df.shape}")
print(f"Default rate (loan_status=1): {df['loan_status'].mean()*100:.2f}%")
print(f"\nColumn types:\n{df.dtypes}")
```

```
Raw shape: (32581, 12)
Clean shape: (32409, 12)
Default rate (loan_status=1): 21.87%

Column types:
person_age                    int64
person_income                 int64
person_home_ownership           str
person_emp_length           float64
loan_intent                     str
loan_grade                      str
loan_amnt                     int64
loan_int_rate               float64
loan_status                   int64
loan_percent_income         float64
cb_person_default_on_file       str
cb_person_cred_hist_length    int64
dtype: object
```

1.2 2. Setup: Classification Target and Encoded Features

[4]:
```
numeric_features = ['person_income', 'person_age', 'person_emp_length',
                    'loan_amnt', 'loan_percent_income', 'loan_int_rate']
categorical_features = ['loan_intent', 'loan_grade',
                        'person_home_ownership', 'cb_person_default_on_file']
```

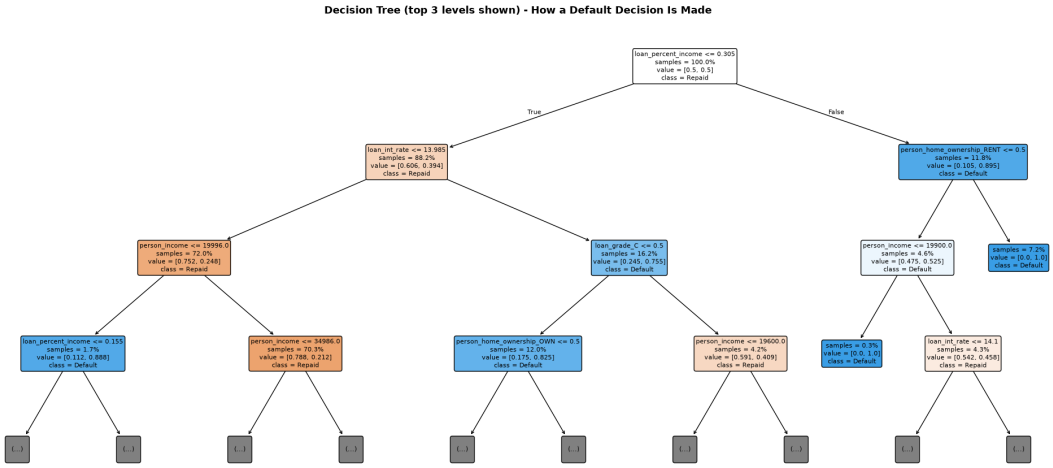
```
X = df[numeric_features + categorical_features].copy()
y = df['loan_status'].copy()
mask = X.notna().all(axis=1)
X, y = X[mask], y[mask]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Trees are scale-invariant (splits are thresholds, not distances), so unlike␣
 ↪Weeks 4-5 we do
# NOT need to scale. We still one-hot encode categoricals so every column is a␣
 ↪usable split.
encoder = ColumnTransformer([
    ('num', 'passthrough', numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])
X_train_enc = encoder.fit_transform(X_train)
X_test_enc = encoder.transform(X_test)
feat_names = numeric_features + encoder.named_transformers_['cat'] \
    .get_feature_names_out(categorical_features).tolist()

print(f"Encoded features: {len(feat_names)} | Train: {len(X_train):,} | Test:␣
 ↪{len(X_test):,}")
print(f"Default rate: {y.mean()*100:.1f}%")
print()
print("Interpretation: Note what we DON'T do - no scaling. A tree asks 'is␣
 ↪income > 50000?',")
print("a question whose answer is unchanged by rescaling. That scale-invariance␣
 ↪is one of the")
print("quiet practical advantages of trees over the SVM and logistic models of␣
 ↪Weeks 4-5.")
```

```
Encoded features: 25 | Train: 25,927 | Test: 6,482
Default rate: 21.9%

Interpretation: Note what we DON'T do - no scaling. A tree asks 'is income >
50000?',
a question whose answer is unchanged by rescaling. That scale-invariance is one
of the
quiet practical advantages of trees over the SVM and logistic models of Weeks
4-5.
```

1.3 3. A Single Decision Tree: Rules You Can Read

[5]:
```
# A shallow tree first, so the rules are legible. class_weight handles the␣
 ↪imbalance.
tree = DecisionTreeClassifier(max_depth=4, class_weight='balanced',␣
 ↪random_state=42)
tree.fit(X_train_enc, y_train)
proba_tree = tree.predict_proba(X_test_enc)[:, 1]
auc_tree = roc_auc_score(y_test, proba_tree)
print(f"Decision Tree (depth 4) Test AUC: {auc_tree:.4f}")
print()
print(classification_report(y_test, tree.predict(X_test_enc),
                            target_names=['Repaid', 'Default']))

fig, ax = plt.subplots(figsize=(20, 9))
plot_tree(tree, feature_names=feat_names, class_names=['Repaid', 'Default'],
          filled=True, rounded=True, fontsize=8, max_depth=3, ax=ax,
          impurity=False, proportion=True)
ax.set_title('Decision Tree (top 3 levels shown) - How a Default Decision Is␣
 ↪Made',
             fontweight='bold', fontsize=13)
plt.tight_layout()
plt.show()

print("Interpretation: Every path from root to leaf is a plain-language␣
 ↪underwriting rule")
print("(e.g. 'high debt burden AND high rate -> likely default'). This␣
 ↪readability is the")
print("tree's headline strength - a declined applicant could be shown the exact␣
 ↪rule path.")
```

```
Decision Tree (depth 4) Test AUC: 0.8695

              precision    recall  f1-score   support

      Repaid       0.93      0.94      0.93      5064
     Default       0.76      0.73      0.75      1418

    accuracy                           0.89      6482
   macro avg       0.84      0.83      0.84      6482
weighted avg       0.89      0.89      0.89      6482
```

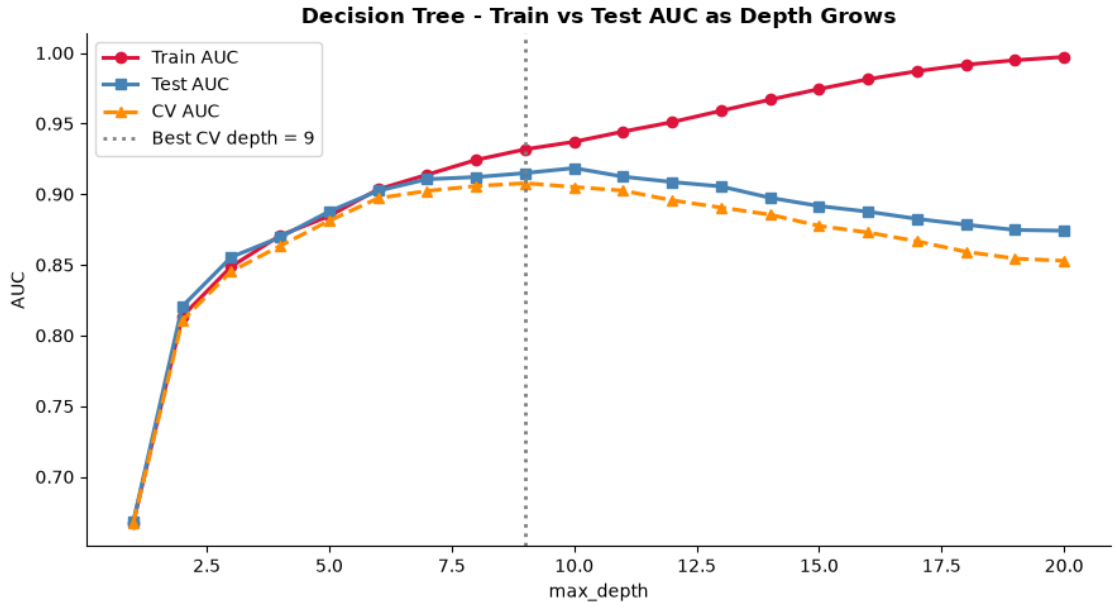


Interpretation: Every path from root to leaf is a plain-language underwriting rule
(e.g. 'high debt burden AND high rate -> likely default'). This readability is the
tree's headline strength - a declined applicant could be shown the exact rule path.

1.4 4. Depth as a Regularizer: Watching a Tree Overfit

[6]:
```
# Sweep max_depth and plot TRAIN vs TEST AUC. The gap between them IS␣
 ↪overfitting made
# visible - the single most important diagnostic for any tree-based model.
depths = range(1, 21)
rows = []
for d in depths:
    t = DecisionTreeClassifier(max_depth=d, class_weight='balanced',␣
 ↪random_state=42)
    t.fit(X_train_enc, y_train)
    rows.append({
        'depth': d,
        'Train AUC': roc_auc_score(y_train, t.predict_proba(X_train_enc)[:, 1]),
        'Test AUC':  roc_auc_score(y_test,  t.predict_proba(X_test_enc)[:, 1]),
        'CV AUC':    cross_val_score(t, X_train_enc, y_train, cv=4,␣
 ↪scoring='roc_auc').mean()
    })
depth_df = pd.DataFrame(rows)
best_depth = int(depth_df.loc[depth_df['CV AUC'].idxmax(), 'depth'])
```

```
fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(depth_df['depth'], depth_df['Train AUC'], marker='o', color='crimson',
        linewidth=2.2, label='Train AUC')
ax.plot(depth_df['depth'], depth_df['Test AUC'], marker='s', color='steelblue',
        linewidth=2.2, label='Test AUC')
ax.plot(depth_df['depth'], depth_df['CV AUC'], marker='^', color='darkorange',
        linewidth=2.2, linestyle='--', label='CV AUC')
ax.axvline(best_depth, color='gray', linestyle=':', linewidth=2,
           label=f'Best CV depth = {best_depth}')
ax.set_xlabel('max_depth')
ax.set_ylabel('AUC')
ax.set_title('Decision Tree - Train vs Test AUC as Depth Grows',␣
 ↪fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print(f"Best depth by CV: {best_depth}")
print("Interpretation: Train AUC marches toward 1.0 as the tree memorizes the␣
 ↪data, but test")
print("AUC peaks and then DECAYS - the widening gap is textbook overfitting.␣
 ↪Capping max_depth")
print("is regularization for trees, the same bias-variance lever as alpha (Wk2)␣
 ↪and C (Wk4-5).")
```



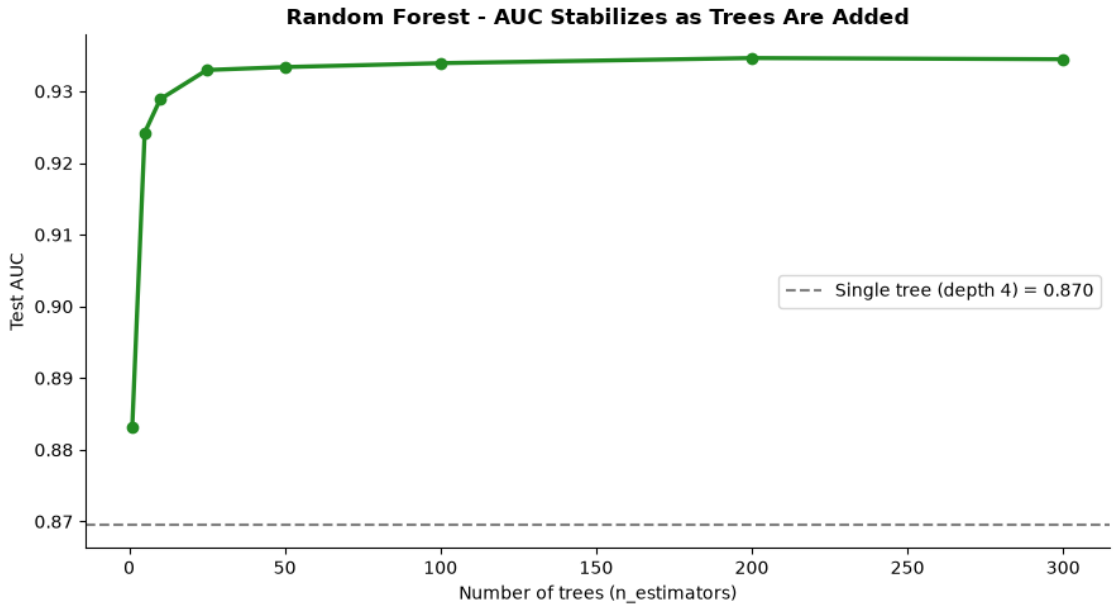
```
Best depth by CV: 9
Interpretation: Train AUC marches toward 1.0 as the tree memorizes the data, but
test
AUC peaks and then DECAYS - the widening gap is textbook overfitting. Capping
max_depth
is regularization for trees, the same bias-variance lever as alpha (Wk2) and C
(Wk4-5).
```

1.5 5. Random Forest: Averaging Away the Variance

[7]:
```
# A random forest grows many trees on bootstrap samples, each split seeing a␣
 ↪random feature
# subset. Averaging their votes cancels the single tree's variance. OOB score␣
 ↪is a built-in
# validation estimate from the ~37% of rows each tree never trained on - free,␣
 ↪no holdout cost.
rf = RandomForestClassifier(
    n_estimators=300, max_depth=None, min_samples_leaf=5,
    class_weight='balanced', oob_score=True, n_jobs=-1, random_state=42)
rf.fit(X_train_enc, y_train)
proba_rf = rf.predict_proba(X_test_enc)[:, 1]
auc_rf = roc_auc_score(y_test, proba_rf)

print("Random Forest (300 trees)")
print(f"  OOB score      : {rf.oob_score_:.4f}")
print(f"  Test AUC       : {auc_rf:.4f}")
print(f"  vs single tree : {auc_tree:.4f}  ->  gain of {auc_rf - auc_tree:+.4f}␣
 ↪AUC")
print()
print(classification_report(y_test, rf.predict(X_test_enc),
                            target_names=['Repaid', 'Default']))

# n_estimators stability curve
ns = [1, 5, 10, 25, 50, 100, 200, 300]
auc_by_n = []
for n in ns:
    r = RandomForestClassifier(n_estimators=n, min_samples_leaf=5,
                               class_weight='balanced', n_jobs=-1,␣
 ↪random_state=42)
    r.fit(X_train_enc, y_train)
    auc_by_n.append(roc_auc_score(y_test, r.predict_proba(X_test_enc)[:, 1]))

fig, ax = plt.subplots(figsize=(9, 5))
ax.plot(ns, auc_by_n, marker='o', color='forestgreen', linewidth=2.4)
ax.axhline(auc_tree, color='gray', linestyle='--', linewidth=1.5,
           label=f'Single tree (depth 4) = {auc_tree:.3f}')
ax.set_xlabel('Number of trees (n_estimators)')
```

```
ax.set_ylabel('Test AUC')
ax.set_title('Random Forest - AUC Stabilizes as Trees Are Added',␣
 ↪fontweight='bold')
ax.legend()
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print("Interpretation: AUC climbs then flattens - past ~100 trees, more trees␣
 ↪add compute, not")
print("accuracy. The forest comfortably clears the single tree because␣
 ↪averaging de-correlated")
print("trees cancels their individual quirks. OOB tracking test AUC confirms we␣
 ↪are not overfit.")
```

```
Random Forest (300 trees)
  OOB score      : 0.9156
  Test AUC       : 0.9345
  vs single tree : 0.8695  ->  gain of +0.0649 AUC

              precision    recall  f1-score   support

      Repaid       0.94      0.96      0.95      5064
     Default       0.84      0.77      0.81      1418

    accuracy                           0.92      6482
   macro avg       0.89      0.87      0.88      6482
weighted avg       0.92      0.92      0.92      6482
```

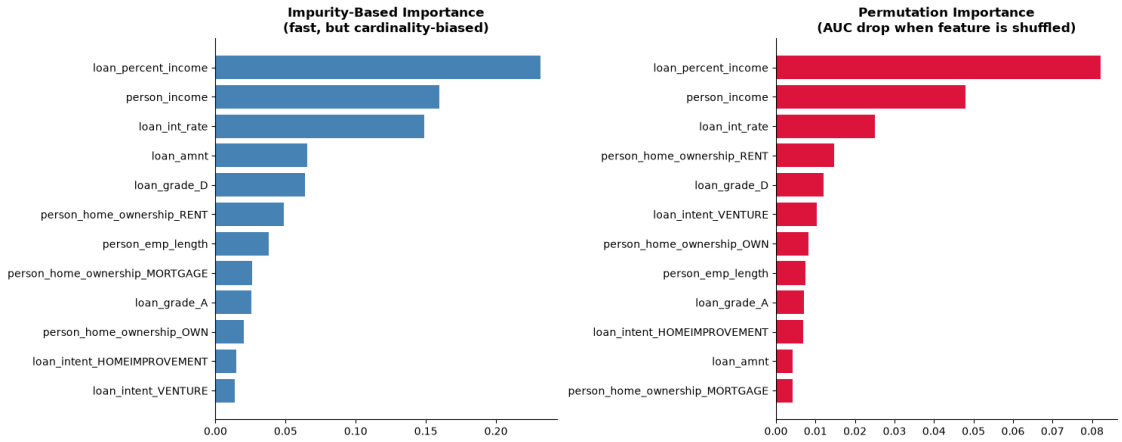


Interpretation: AUC climbs then flattens - past ~100 trees, more trees add compute, not
accuracy. The forest comfortably clears the single tree because averaging de-correlated
trees cancels their individual quirks. OOB tracking test AUC confirms we are not overfit.

1.6 6. Feature Importance: Impurity vs Permutation

[8]:
```
# Default (impurity) importance is fast but biased toward high-cardinality␣
 ↪features.
# Permutation importance is the honest cross-check: shuffle one feature,␣
 ↪measure how much
# test AUC drops. A feature that matters will hurt when scrambled.
imp_gini = pd.DataFrame({'Feature': feat_names,
                         'Impurity': rf.feature_importances_})
perm = permutation_importance(rf, X_test_enc, y_test, scoring='roc_auc',
                              n_repeats=8, random_state=42, n_jobs=-1)
imp_perm = pd.DataFrame({'Feature': feat_names,
                         'Permutation': perm.importances_mean})
imp = imp_gini.merge(imp_perm, on='Feature')

top = imp.sort_values('Permutation', ascending=False).head(12)
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
g = top.sort_values('Impurity')
axes[0].barh(g['Feature'], g['Impurity'], color='steelblue')
```

```
axes[0].set_title('Impurity-Based Importance\n(fast, but cardinality-biased)',␣
  ↪fontweight='bold')
p = top.sort_values('Permutation')
axes[1].barh(p['Feature'], p['Permutation'], color='crimson')
axes[1].set_title('Permutation Importance\n(AUC drop when feature is␣
  ↪shuffled)', fontweight='bold')
for ax in axes:
    ax.spines['top'].set_visible(False); ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print("Top default drivers by permutation importance:")
print(top.sort_values('Permutation', ascending=False)[['Feature',␣
  ↪'Permutation']]
      .head(8).to_string(index=False))
print()
print("What this means for the Chief Risk Officer: the features that survive␣
  ↪the permutation")
print("test are the ones the bank should monitor most closely - they carry␣
  ↪real, non-redundant")
print("signal about default. Reassuringly, they echo the odds-ratio drivers␣
  ↪from Week 4's")
print("logistic model, so two very different model families agree on the␣
  ↪underlying risk story.")
```




```
Top default drivers by permutation importance:
                  Feature  Permutation
       loan_percent_income     0.082257
             person_income     0.047891
             loan_int_rate     0.025031
person_home_ownership_RENT     0.014768
```

```
              loan_grade_D     0.011980
      loan_intent_VENTURE      0.010339
person_home_ownership_OWN      0.008142
        person_emp_length      0.007415
```

What this means for the Chief Risk Officer: the features that survive the permutation
test are the ones the bank should monitor most closely - they carry real, non-redundant
signal about default. Reassuringly, they echo the odds-ratio drivers from Week 4's
logistic model, so two very different model families agree on the underlying risk story.

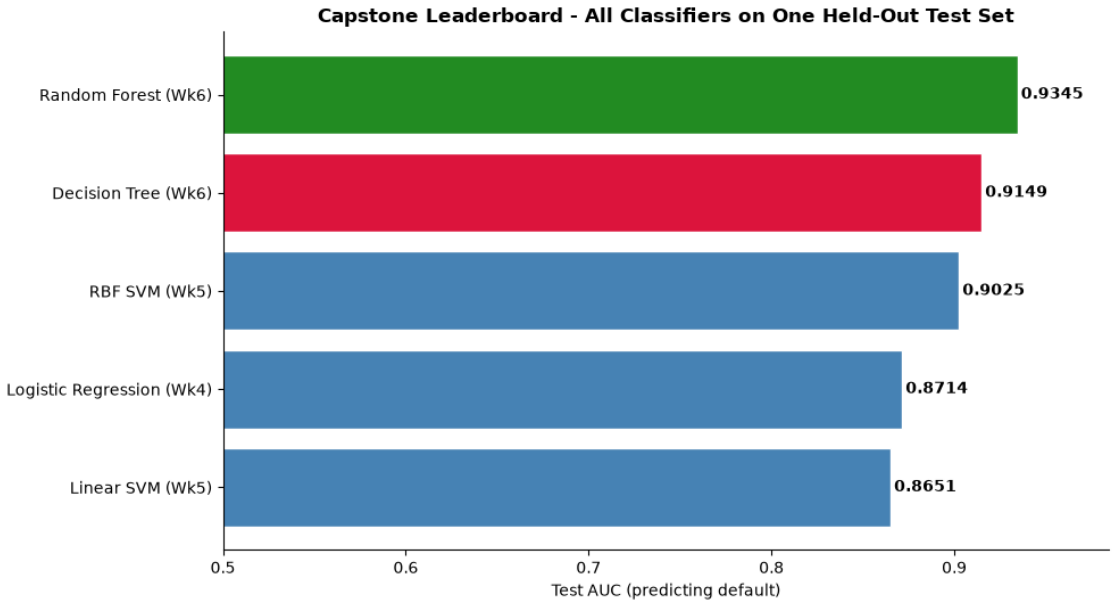
1.7 7. The Capstone Leaderboard: Every Classifier, Head-to-Head

[9]:
```
# Self-contained re-fit of each model family from Weeks 4-6 on THIS train/test␣
 ↪split,
# so the comparison is apples-to-apples. (Logistic & SVM need scaling; trees do␣
 ↪not.)
scaled_prep = ColumnTransformer([
    ('num', StandardScaler(), numeric_features),
    ('cat', OneHotEncoder(handle_unknown='ignore', sparse_output=False),␣
 ↪categorical_features)
])

# SVM is O(n^2); fit it on a stratified subsample for tractability (as in Week␣
 ↪5).
sub_n = min(6000, len(X_train))
X_sub, _, y_sub, _ = train_test_split(X_train, y_train, train_size=sub_n,
                                      random_state=42, stratify=y_train)

contenders = {
    'Logistic Regression (Wk4)': Pipeline([('p', scaled_prep),
        ('m', LogisticRegression(max_iter=1000, class_weight='balanced'))]),
    'Linear SVM (Wk5)': Pipeline([('p', scaled_prep),
        ('m', SVC(kernel='linear', class_weight='balanced', probability=True,␣
 ↪random_state=42))]),
    'RBF SVM (Wk5)': Pipeline([('p', scaled_prep),
        ('m', SVC(kernel='rbf', gamma='scale', class_weight='balanced',
                  probability=True, random_state=42))]),
    'Decision Tree (Wk6)': Pipeline([('p', encoder),
        ('m', DecisionTreeClassifier(max_depth=best_depth,␣
 ↪class_weight='balanced',
                                     random_state=42))]),
```

```
    'Random Forest (Wk6)': Pipeline([('p', encoder),
        ('m', RandomForestClassifier(n_estimators=300, min_samples_leaf=5,
                                     class_weight='balanced', n_jobs=-1,␣
 ↪random_state=42))]),
}

board = []
for name, model in contenders.items():
    if 'SVM' in name:
        model.fit(X_sub, y_sub)        # subsample for the quadratic-cost models
    else:
        model.fit(X_train, y_train)
    proba = model.predict_proba(X_test)[:, 1]
    board.append({'Model': name, 'Test AUC': round(roc_auc_score(y_test,␣
 ↪proba), 4)})

board_df = pd.DataFrame(board).sort_values('Test AUC', ascending=False).
 ↪reset_index(drop=True)
print(board_df.to_string(index=False))

fig, ax = plt.subplots(figsize=(10, 5.5))
order = board_df.sort_values('Test AUC')
colors = ['forestgreen' if 'Forest' in m else 'crimson' if 'Tree' in m
          else 'steelblue' for m in order['Model']]
ax.barh(order['Model'], order['Test AUC'], color=colors, edgecolor='white')
for i, v in enumerate(order['Test AUC']):
    ax.text(v + 0.002, i, f'{v:.4f}', va='center', fontweight='bold')
ax.set_xlabel('Test AUC (predicting default)')
ax.set_title('Capstone Leaderboard - All Classifiers on One Held-Out Test Set',
             fontweight='bold')
ax.set_xlim(0.5, max(board_df['Test AUC']) + 0.05)
ax.spines['top'].set_visible(False)
ax.spines['right'].set_visible(False)
plt.tight_layout()
plt.show()

print()
print("What this means for the Chief Risk Officer: this single chart is the␣
 ↪deliverable that")
print("matters. It ranks every approach the project tried on identical, honest␣
 ↪footing. Where")
print("the random forest leads, it earns deployment IF the bank can accept␣
 ↪reduced")
print("interpretability; where logistic regression is within a whisker, its␣
 ↪transparent odds")
```

```
print("ratios may be worth the small accuracy cost in a regulated lending␣
  ↪decision. The right")
print("model is the one whose accuracy/interpretability trade-off matches the␣
  ↪business risk.")
```

```
                  Model  Test AUC
      Random Forest (Wk6)    0.9345
      Decision Tree (Wk6)    0.9149
            RBF SVM (Wk5)    0.9025
Logistic Regression (Wk4)    0.8714
         Linear SVM (Wk5)    0.8651
```



What this means for the Chief Risk Officer: this single chart is the deliverable that
matters. It ranks every approach the project tried on identical, honest footing. Where
the random forest leads, it earns deployment IF the bank can accept reduced
interpretability; where logistic regression is within a whisker, its transparent odds
ratios may be worth the small accuracy cost in a regulated lending decision. The right
model is the one whose accuracy/interpretability trade-off matches the business
risk.

1.7.1 Week 6 Takeaways and Closing the Capstone Loop

What this week established: - A **single decision tree** turns underwriting into readable if/then rules and needs no feature scaling, but overfits sharply as depth grows - shown directly by the widening train-vs-test AUC gap, with `max_depth` as its regularizer. - A **random forest** averages hundreds of de-correlated trees to recover stability and accuracy; its **OOB score** is a free validation estimate, and AUC flattens once enough trees are added. - **Permutation importance** is the honest importance measure, and it agrees with Week 4's logistic odds ratios - two different model families telling the same risk story. - The **leaderboard** ranks every classifier from Weeks 4-6 on one test set, reframing model choice as an explicit accuracy-vs-interpretability trade-off rather than a horse race.